

Machine Learning-Driven Prediction of Phase Structures in High-Entropy Alloys for Aerospace Applications

Summary

High-entropy alloys (HEAs) are a novel type of alloy composed of five or more elements in near-equiatomic ratios. They offer exceptional strength, thermal stability, and corrosion resistance, making them promising for aerospace applications. However, their vast compositional space ($\approx 10^8$ combinations) and the high cost of traditional experimentation or simulation hinder rapid discovery.

This dissertation develops a modular machine learning framework to predict HEA phases (FCC, BCC, or multiphase/intermetallic). The framework supports automated data encoding, feature selection, and hyperparameter optimisation algorithms.

Datasets contained elemental compositions and mechanical properties. Three datasets were collected and augmented using a custom algorithm to impute missing values, resulting in 85996 complete samples for model training, with added thermodynamic features that were computed from atomic and compositional data to enlarge the feature set and improve predictive power.

Random Forest, XGBoost, and Multilayer Perceptron classifiers achieved baseline accuracies of 76.1%, 75.7%, and 73.9%, respectively. The addition of thermodynamic fields, feature selection, and hyperparameter optimisation and data imputation raised predictive accuracy to 90.06%.

This work shows that an efficiently designed ML pipeline can offer a high accuracy alternative to phase prediction in minimal time (115.08 seconds for a training cycle), reducing reliance on costly trial-and-error methods. By enabling rapid virtual screening of unexplored compositions, this approach accelerates the development of advanced HEAs for high-performance aerospace use.

However, limitations remain: the reliance on imputed data introduces uncertainty, performance may degrade on underrepresented compositions and datasets were unable to capture concepts such as processing-history and microstructural descriptors. These drawbacks highlight the need for expanded, higher-fidelity datasets in future HEA design studies.

Contents

Summary	1
Contents	2
Nomenclature	4
Acronyms	5
1. Introduction	6
2. Background and Context	6
2.1 Desirable Material Properties of HEAs	7
2.2 Challenges Surrounding Phase Formation of HEAs	7
2.3 Issues with conventional testing methods	8
2.4 Advantages of Data-Driven Approaches	8
2.5 Aims and Objectives	8
3. Literature Review	9
3.1 Prior Studies and Challenges in HEA Phase Prediction	9
3.2 Selecting Model Architectures	10
3.3 Improving Model Accuracy with Small Datasets	10
3.3.1 Transfer Learning	10
3.3.2 Thermodynamic Fields	11
3.3.3 Crude Estimation of Property	11
3.4 Optimisation algorithms	12
3.4.1 Feature Selection	12
3.4.2 Hyperparameter Optimisation	12
3.5 Key Takeaways from the Literature Review Section	12
4. Methodology	13
4.1 Objectives for model development	13
4.2 Finding Datasets	14
4.2.1 Preprocessing of data	15
4.3 Workflow of the application	16
4.4 Machine Learning Model Selection	16
4.4.1 Advantage of using multiple architectures	18
4.5 Designing the Model Class	18
4.6 Thermodynamic fields	19

4.7 Optimisation Methods.....	20
4.7.1 Feature Selection.....	20
4.7.1.1 LASSO Regression	21
4.7.1.2 Genetic algorithms	22
4.7.1.3 Elliot Feature Selection	23
4.7.2 Hyperparameter Optimisation	24
4.7.2.1 Bayesian Search.....	25
4.7.2.2 Parallel Grid Search.....	26
4.7.2.3 Genetic algorithm.....	27
4.8 Crude Estimation of Property.....	28
5. Results, Analysis and Discussion.....	30
5.1 Baseline Model accuracy	30
5.2 Assessment of algorithms.....	32
5.2.1 Thermodynamic Fields	32
5.2.2 Feature Selection	34
5.2.3 Hyperparameter Optimisation.....	37
5.2.4 Crude estimation of property	39
5.3 Building an optimal model	41
5.3.1 Model architecture	41
5.3.2 Model Training.....	41
5.4 Model Analysis	41
5.4.1 Model Accuracy	42
5.4.2 Robustness testing with added noise	43
5.4.3 Distribution Alignment Using Kolmogorov–Smirnov Test	44
5.4.4 Model Calibration with Brier Score and Calibration Curve.....	45
5.5 Discussion on project success and revisiting aims	46
5.6 Model Limitations	46
6. Conclusions.....	47
7. Future Work	47
References.....	48
Appendix – Algorithm Pseudocode	52
Model Class.....	52

Genetic Feature Selection	53
Bayesian Search	54
Grid Search	55
Genetic Hyperparameter Optimisation	56

Nomenclature

Symbol	Quantity	Unit
ΔG_{mix}	Gibbs Free Energy of Mixing	J/mol
ΔH_{mix}	Mixing Enthalpy	J
ΔS_{mix}	Mixing Entropy	J/mol K
T	Temperature	K
ΔG_{f}	Gibbs Free Energy to form an Intermetallic	J/mol
ΔH_{f}	Mixing Enthalpy to form an Intermetallic	J
ΔS_{f}	Mixing Entropy to form an Intermetallic	J/mol K
R	Gas Constant	J/mol K
VEC	Valence Electron Concentration	-
$\Delta\chi$	Electronegativity Difference	-
δ	Atomic Size Difference	m
x_i	Molar fraction of element x.	-
c_i	Atomic percentage of the <i>ith</i> alloying element	-
c_j	Atomic percentage of the <i>jth</i> alloying element	-
T_m	Mixing temperature	K
VEC_i	VEC of the <i>ith</i> element	-
χ_i	Electronegativity of the <i>ith</i> element	-
r_i	Atomic radius of the <i>ith</i> element	m
λ	Regularisation value	-
β_j	Weighting for predictor	-
BS_c	Brier Score	-
N	Number of samples	-

p_i	Predicted probability for sample i	-
y_i	True outcome for sample i	-

Acronyms

Acronym	Meaning
HEA	High-Entropy Alloy
MPEA	Multi-Principal Element Alloy
ML	Machine Learning
RF	Random Forest
XGB	eXtreme Gradient Boosting (XGBoost)
MLP	Multilayer Perceptron
GA	Genetic Algorithm
SMBO	Sequential Model-Based Optimisation
LASSO	Least Absolute Shrinkage and Selection Operator
CEP	Crude Estimation of Property
DFT	Density Functional Theory
MD	Molecular Dynamics
KNN	K-Nearest Neighbours
LR	Logistic Regression
DT	Decision Tree
NB	Naive Bayes
HV	Vicker's hardness
SVM	Support Vector Machine
ANN	Artificial Neural Network
FCC	Face-centred Cubic
BCC	Body-Centred Cubic
IM	Intermetallic
VEC	Valence Electron Concentration
RMSE	Root Mean Squared Error

1. Introduction

Alloy development dates back to early civilisation, traditionally involving the addition of small amounts of alloying elements to a base metal to enhance properties—a method still prevalent in aerospace with aluminium, titanium, nickel, and steel alloys [1]. However, this approach restricts compositional diversity.

In 2004, Yeh et al. [2] proposed the concept of High-Entropy Alloys (HEAs), which mark a major shift in materials science by combining five or more principal elements in near-equal proportions [1], leading to high mixing entropies that stabilise solid solutions and enhance properties like strength, thermal stability, and corrosion resistance [2].

It is important for HEAs to form a single-phase solution, and compositions that form intermetallic phases is undesirable [1]. Their vast compositional space—and the fact that single phase formations only take up $\sim 5\%$ of this space—makes traditional experimental and computational methods impractical due to high time and resource costs [3], [4].

This dissertation addresses these challenges by using machine learning (ML) to predict HEA phase structures. A dynamic ML framework is developed, incorporating advanced feature engineering, hyperparameter optimisation, and a custom Crude Estimation of Property (CEP) algorithm to handle the issues of data scarcity and optimise model architectures [3], [5], [6].

This project evaluates multiple ML architectures, enhanced by optimisation algorithms. Their ability to solve classification and regression problems is evaluated and model performance is tested in detail. Results show that a well-designed ML pipeline can achieve high prediction accuracy of 90.06%, revealing key factors that influence phase formation, offering a powerful new approach for rapid, cost-effective alloy design.

2. Background and Context

This section outlines the advantages and challenges of HEAs, and the limitations of conventional testing methods, and proposes machine learning as a potential tool enable efficient HEA development.

2.1 Desirable Material Properties of HEAs

Unlike conventional alloys, HEAs have no matrix element; all elements are added in near-equal ratios, creating a strong solution strengthening effect that impedes dislocation motion and improves strength [2]. As shown in Table 1, certain HEAs demonstrate higher hardness than traditional alloys and retain their strength better after annealing.

Certain HEAs provide higher strength-to-weight ratios than conventional alloys, along with improved resistance to oxidation, fatigue, wear, and creep [7]. HEAs have also demonstrated enhanced strength at extreme temperatures [8], [7]. These properties make HEAs highly promising for aerospace applications, particularly in gas turbines, high-stress components, and corrosive environments.

Alloys	Hardness, HV as cast	Hardness, HV annealed
CuTiVFeNiZr	590	600
AlTiVFeNiZr	800	790
MoTiVFeNiZr	740	760
CuTiVFeNiZrCo	640	620
AlTiVFeNiZrCo	790	800
MoTiVFeNiZrCo	790	790
CuTiVFeNiZrCoCr	680	680
MoTiVFeNiZrCoCr	850	850
316 Stainless Steel	189	155
17-4 PH Stainless Steel	410	362
Hastelloy C	236	280
Stellite 6	413	494
Ti-6-Al-4V	412	314

Table 1 - Hardness of as-cast and annealed HEAs compared with conventional alloys. [2]

2.2 Challenges Surrounding Phase Formation of HEAs

In HEAs, the formation of intermetallic phases is undesirable, as it negatively impacts mechanical properties [1]. Predicting whether a HEA will form a single phase depends on thermodynamic principles, particularly Gibbs free energy of mixing. If the free energy of mixing (ΔG_{mix}) is lower than that of intermetallic formation (ΔG_f), single-phase formation is favoured.

$$A + B = AB \text{ (solution)} \Delta G_{mix} = \Delta H_{mix} - T\Delta S_{mix}$$

Equation 1 - Gibbs Free Energy equation to form a solution with components A and B [2]

$$A + B = AB \text{ (solution)} \Delta G_f = \Delta H_f - T\Delta S_f$$

Equation 2 - Gibbs free energy equation to form an intermetallic with components A and B [2]

To simplify prediction, alloys are often assumed to behave ideally. Where $\Delta H_{mix} \approx 0$ and $\Delta S_f \approx 0$. As a result, ΔG_{mix} depends primarily on mixing entropy, which increases with the number of elements.

$$\Delta S_{mix} = -R \sum x_i \ln x_i$$

Equation 3 – Mixing entropy when the solution is assumed to be ideal [2]

Considering Equation 3, the mixing entropy can be simplified to be based on the number of mixing elements.

$$\Delta S_{mix} = R \ln n$$

Equation 4 - relation between number of elements and mixing entropy where n is number of elements [2]

Thus, using five or more elements should, in theory, favour single-phase HEAs [2]. However, experiments showed that most HEAs still formed intermetallics [9], highlighting that

configurational entropy alone cannot accurately predict phase formation. Due to the complex, high-dimensional nature of HEAs, more advanced prediction methods are required.

2.3 Issues with conventional testing methods

HEAs offer desirable properties but are difficult to design due to their vast compositional space, making conventional approaches inefficient [2]. Key limitations of current testing and simulation methods highlight the need for more scalable solutions to accelerate HEA discovery.

With 64 elements yielding nearly 10^8 possible HEA variants, most combinations fail to form desirable single-phase structures [10], [11]. Despite this, HEA research still relies heavily on trial-and-error and computational modelling [12], approaches that are too resource-intensive and inefficient for large-scale discovery.

Only about 5% of the HEA composition space yields stable single-phase FCC or BCC structures [13], [11]. Accurately predicting these phases is complex due to the many interacting variables, making traditional methods poorly suited to such high-dimensional problems.

Density Functional Theory (DFT) and Molecular Dynamics (MD) simulations offer a theoretical alternative but are computationally prohibitive. Simulating a 250-atom Cu-Zn alloy could demand 600,000 calculations which can easily cost 1,000,000 CPU core hours [14]. As the complexity of HEAs increases, simulations require even larger models, making DFT and MD too slow and costly for widespread application.

2.4 Advantages of Data-Driven Approaches

ML offers a promising alternative to this issue by enabling fast, scalable, and accurate phase prediction using compositional, thermodynamic, and mechanical inputs. In fields such as Computational Fluid Dynamics, ML application has already demonstrated significant gains in speed and resolution—achieving up to 8–10× higher resolution and 40–80× faster computation [15].

ML models are particularly well suited to modelling non-linear relationships in systems with many interacting variables. Unlike traditional statistical approaches that rely on user-defined assumptions, ML algorithms learn patterns directly from data, improving accuracy and enabling the discovery of previously unknown relationships [16]. This makes ML especially valuable for HEAs, where the complexity of interactions often exceeds the capabilities of conventional modelling.

2.5 Aims and Objectives

The aim of this project is to develop a robust, high-accuracy, machine learning application capable of predicting the phase structures of HEAs. This tool will support efficient alloy design and provide a potential solution to current experimental challenges.

Objectives:

- Develop a machine learning framework for fast and dynamic model development.
- Design a user-friendly interface to allow researchers to create and optimise their own models.
- Implement and compare various optimisation methods to enhance model accuracy.
- Construct and validate an optimal phase prediction model suitable for engineering applications.

3. Literature Review

This section identifies the challenges surrounding design of a ML model to predict phase in HEAs, investigates ideal model architectures, methods to address limited datasets and complex feature spaces, and investigates effective optimisation methods to improve model performance.

3.1 Prior Studies and Challenges in HEA Phase Prediction

The success of prior studies in applying machine learning to HEA phase prediction confirms the feasibility and value of data-driven approaches. However, their limitations in data volume, completeness, and validation create a clear opportunity for improvement.

Huang et al. [17]. used thermodynamic features on a dataset of 401 HEAs to train K-Nearest Neighbours (KNN), Support Vector Machines (SVM), and Artificial Neural Networks (ANN), achieving prediction accuracies on test sets of 68.6%, 64.3%, and 74.3%, respectively. In another study, I. Peivaste et al. used XGboost and Random Forest, achieving 86% accuracy predicting all phases with a dataset of 5692 experimental records. [13].

Both studies validate the application of machine learning to HEA phase prediction but also reveal three key challenges in model design:

- **Model Architecture Selection:** Choosing the right algorithm is essential to accurately capture the complex, nonlinear relationships between elemental composition and phase structure [18]. Options like decision trees, SVMs, or neural networks each offer different advantages in terms of interpretability, flexibility, and performance [18].
- **Limited Datasets:** HEA datasets are often small and imbalanced, with certain phases like FCC or BCC being overrepresented [6], [17]. This can lead to overfitting, biased predictions, and reduced model reliability, especially for rarer phase classes.
- **Suboptimal Parameters and Feature Sets:** Poorly chosen features or hyperparameters can significantly limit model performance. Effective feature engineering is critical, as is hyperparameter tuning to ensure proper learning without under or overfitting [19].

3.2 Selecting Model Architectures

The task of predicting alloy phase structures can be approached as a multiclass classification problem, where a machine learning model outputs the most likely phase for a given composition [20].

Choosing architectures that have demonstrated high accuracy on similar problems is key to achieving reliable predictions for phase formation. A. Hassan et al. tested various models on a multiclass classification task and found that the random forest algorithm yielded the highest accuracy on test data [18]. Their results showed that tree-based and regression models outperformed traditional approaches like KNN (Figure 1).

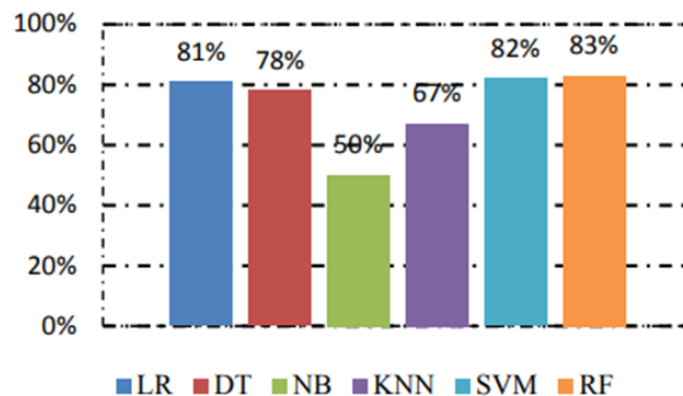


Figure 1 - Accuracy of Classification Models for a multiclass classification problem [18]

3.3 Improving Model Accuracy with Small Datasets

A major challenge in applying to materials science is the limited availability of high-quality data. In fields like HEA design, datasets are not only small but also diverse, making it difficult for models to learn reliable patterns [20], [14].

As shown in Figure 2, model accuracy tends to improve with more training data, but in materials science, generating additional data requires more experimentation, which is costly and time-consuming [6]. To address this, alternative strategies must be employed to improve predictive performance without expanding the experimental dataset.

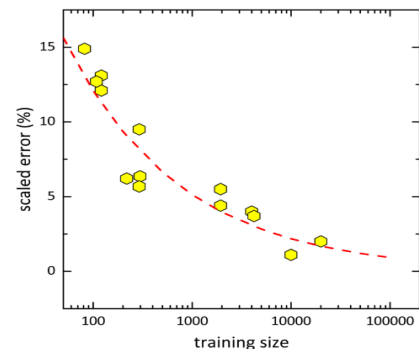


Figure 2 - Inverse relation between scaled error and training size for a classifier [6]

3.3.1 Transfer Learning

Transfer learning provides a practical solution by allowing models to learn from related domains where data is more abundant [21]. Large datasets exist for conventional alloys alongside datasets of DFT simulations of HEAs [4], [22], which can be used to pre-train models before fine-tuning them on the limited HEA-specific data. This approach reduces dependency on new experiments and can improve classification performance on unseen data.

However, care must be taken to manage issues such as domain mismatch or negative transfer, where knowledge from the source domain may not translate effectively to the target, which

can result in a loss of accuracy [21]. These challenges can be addressed through careful model validation and by combining transfer learning with optimisation and augmentation techniques to refine predictions.

3.3.2 Thermodynamic Fields

Enhancing the feature space with calculated thermodynamic properties significantly improves the model's ability to capture underlying phase formation mechanisms. Derived directly from elemental composition, these features offer physically meaningful insights and have been shown to correlate strongly with phase stability and transitions [23]. Key descriptors include:

1. **Mixing Entropy (ΔS_{mix}) and Enthalpy (ΔH_{mix}):** Reflect the degree of disorder and interaction energy. High entropy stabilises solid solutions, while negative enthalpy values may signal intermetallic formation [23].
2. **Valence Electron Concentration (VEC):** Predicts phase structure; higher values generally favour FCC, while lower values favour BCC [23].
3. **Electronegativity Difference ($\Delta\chi$):** Captures bonding asymmetry and its role in promoting multi-phase structures [23].
4. **Atomic Size Difference (δ):** Indicates lattice distortion, which can enhance single-phase stability [23].

These descriptors, computed from known elemental properties and composition ratios, enrich the dataset without requiring new experiments, improving classification performance without requiring additional experimentation.

3.3.3 Crude Estimation of Property

In high-dimensional compositional datasets, missing values are common and can significantly reduce usable sample size. Crude estimation of Property (CEP) addresses this by generating low-fidelity approximations of target properties based on simplified models or empirical rules [6]. While these estimates do not fully reflect complex atomic interactions, they serve as statistically valid surrogates for missing values. Their inclusion brings two major benefits:

1. **Enhanced Data Completeness:** By imputing missing entries, CEP maximises the number of complete samples available for training, enabling broader feature inclusion [6].
2. **Improved Accuracy and Reduced Complexity:** The surrogate values act as baseline estimates, allowing the model to focus on learning deviations from the crude prediction. This approach reduces the model's effective degrees of freedom, helping to prevent underfitting on small datasets [6].

Together, the integration of transfer learning, thermodynamic features and CEP provides a robust framework for improving model accuracy in data-scarce environments. By leveraging theoretical insights and surrogate modelling, this dual approach helps overcome common limitations in HEA research and enables more reliable ML-driven phase prediction.

3.4 Optimisation algorithms

Model optimisation is critical for improving prediction accuracy, especially when handling the small, noisy datasets which are found in HEA research. Feature selection and hyperparameter optimisation have both been identified as fast, and effective methods to improve model stability and accuracy in classification tasks [19].

3.4.1 Feature Selection

Feature selection plays a critical role in improving model performance by identifying the most relevant features for training and prediction. In classification problems, datasets often contain redundant or irrelevant features that add noise, increase computational cost, and reduce accuracy. A large feature vector with unnecessary inputs can lead to overfitting, where the model fits to patterns that do not apply well to unseen data. By removing these redundant features, feature selection improves both model efficiency and predictive accuracy [5].

For phase prediction tasks, embedded methods and optimisation-based techniques are particularly effective. Embedded methods—such as LASSO regression and tree-based feature importance—perform feature selection during model training, ensuring relevance without requiring a separate selection step [5]. Optimisation-based approaches, such as genetic algorithms, are well-suited for navigating high-dimensional feature spaces [24]. These methods are especially useful when dealing with complex datasets, offering a strategic way to refine input variables and boost model performance.

3.4.2 Hyperparameter Optimisation

Hyperparameters are external settings used to control the training process of a machine learning model, such as learning rate, tree depth, or the number of estimators [25]. In the context of HEA phase prediction, optimising these parameters is essential for maximising model accuracy and generalisation. Two powerful approaches for hyperparameter tuning are Genetic Algorithms (GAs) and Sequential Model-Based Optimisation (SMBO) [25].

GAs apply evolutionary principles to efficiently search high-dimensional parameter spaces [26], striking a balance between performance and computational cost. They are particularly effective when the search space is large and complex.

SMBO methods—such as Bayesian Optimisation—build a probabilistic model (often Gaussian) of the objective function and use it to iteratively select promising hyperparameter combinations [27]. This approach is ideal for scenarios where training is computationally expensive, as it intelligently balances exploration and exploitation to converge on optimal settings more efficiently than brute-force methods like grid search.

Combining GAs and SMBO can further enhance optimisation robustness by leveraging the global search capabilities of GAs with the adaptive precision of SMBO, resulting in more accurate and efficient model tuning.

3.5 Key Takeaways from the Literature Review Section

Machine learning presents a powerful approach for high-entropy alloy (HEA) design, offering the ability to accurately predict phase formation while significantly reducing the cost

and time associated with experimental work [14]. However, several challenges remain, including small and incomplete datasets, as well as the use of suboptimal features and model architectures.

To address these limitations, strategies such as additional fields, data imputation using Crude Estimation of Property (CEP), feature selection methods, and hyperparameter optimisation can be used to improve model performance. Additionally, transfer learning, which leverages larger datasets from related alloy systems, offers an effective solution to the problem of data scarcity.

Together, these methods provide a robust foundation for improving the accuracy, efficiency, and generalisability of phase prediction models. By integrating these advances, the field moves closer to faster, more precise alloy development, ultimately driving innovation in materials engineering.

4. Methodology

This section covers the workflow used to design an effective machine learning framework to enable accurate predictions of phase in HEAs. Covering data collection, preprocessing, model selection, algorithm design and code efficiency analysis.

4.1 Objectives for model development

The objective of this work is to develop a flexible, scalable framework for testing a wide range of machine learning models on both classification and regression problems. The core focus is to apply advanced optimisation algorithms to maximise phase classification performance. Enabling development of a final high accuracy model to be used to predict phase of HEAs.

The model development pipeline consists of:

- **Data Collection:** Aggregation and preparation of datasets relevant to the prediction tasks, ensuring feature quality and consistency.
- **Model Selection:** Evaluation and selection of suitable machine learning architectures (e.g., tree-based models, neural networks, ensemble methods) tailored to each task.
- **Algorithm Design for Optimisation:** Integration of advanced techniques such as feature selection and hyperparameter optimisation to fine-tune model parameters and enhance performance.
- **Model Testing and Validation:** Comprehensive evaluation of accuracy and generalisability using a train-test split validation method [20].

All code is developed in Python 3.9, chosen for its extensive machine learning and data science libraries. The implementation leverages freely available tools including NumPy, pandas, scikit-learn, XGBoost, TensorFlow, and Matplotlib. NumPy and pandas enable efficient handling of large tabular datasets, critical due to the size of the data. Scikit-learn, XGBoost, and

TensorFlow provide reliable frameworks for constructing, training, and evaluating machine learning models.

The pseudocode for all optimization algorithms is provided either within the main body of the document or in the [appendix](#).

4.2 Finding Datasets

The design of a machine learning algorithm first involves curation of high-quality datasets. The dataset must be accurate, reliable, sufficiently large to reveal underlying patterns, and diverse enough to cover the range of scenarios the model will encounter. In this work, three distinct datasets have been assembled, each contributing unique characteristics to the training and validation process.

Dataset	Description	Number of Entries
Dataset 1	HEA compositions, selected for their verified quality and experimental validation [22]	396
Dataset 2	Multi-Principal Element Alloys (MPEAs), providing additional compositional variations and broader feature diversity [28]	1546
Dataset 3	Estimated values for MPEAs and HEAs calculated via Density Functional Theory (DFT) [4]	84,024

Table 2 - Three primary datasets collected for training and validation of the ML model

This configuration ensures the model benefits from both high-quality, experimentally validated data and a diverse, extensive training base, balancing comprehensive feature information with adequate sample size. Datasets 1 and 2 will contribute to primary training data. Dataset 3 is used for pre-training, exposing the model to a broader range of compositional variations and improving general predictive capabilities with transfer learning.

A key challenge across all datasets is missing entries (shown below in Figure 3), forcing a trade-off between the number of training instances and the number of available features. Considering only complete entries would significantly reduce dataset size and eliminate variability. Conversely, reducing feature sets to retain more entries can limit the model's ability to recognise patterns, potentially harming accuracy.

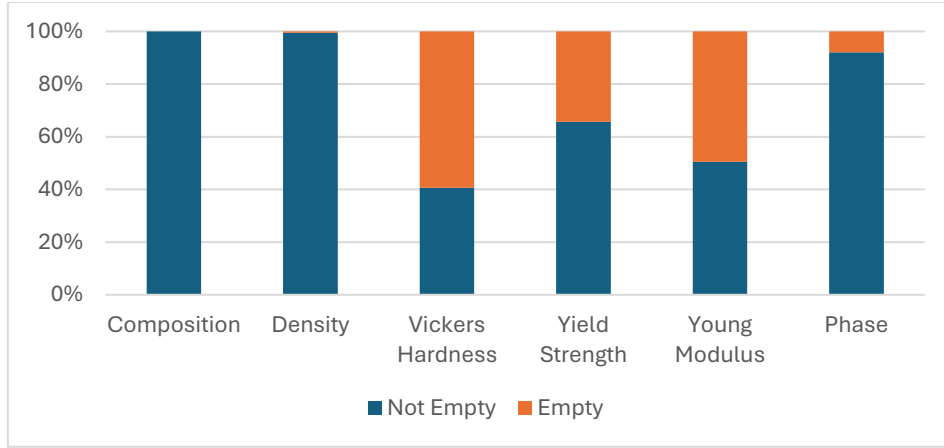


Figure 3 – Distribution of empty regions in each feature

4.2.1 Preprocessing of data

The current datasets are structured with various fields capturing different aspects of the alloys' properties and compositions. They include both numerical measurements, such as density and hardness, and categorical data, such as composition and phase classification. Table 3 provides an example of the dataset structure.

<i>composition</i>	<i>phase</i>	<i>density</i>	<i>hv</i>	<i>yield_strength</i>	<i>young_modulus</i>
<i>CoFeNi</i>	FCC	8.5	125	204	207
<i>CoFeNiSi0.25</i>	FCC	7.7	149	196	194
<i>CoFeNiSi0.5</i>	FCC + Im	7.1	287	476	Empty
<i>CoFeNiSi0.75</i>	FCC + Im	6.6	570	1301	Empty

Table 3 – Entries from Dataset 1 (specified in Table 2)

Since machine learning models require numerical input features and targets, restructuring string-valued fields is required. The Composition field is converted into numerical fractions for each element, with each element becoming an individual feature. Enabling the model to process composition data as continuous variables. However, this method greatly expands the feature set, and creates a potential issue of high dimensionality, which could reduce accuracy. This problem can be resolved with feature selection algorithms, reducing the total number of input features and removing the issue of high dimensionality.

Similarly, the Phase field is numerically encoded. Selected phases (FCC, BCC, and "other" — covering multiphase and intermetallic structures) are mapped to distinct numerical labels, making phase information usable in regression tasks.

Co	Cr	Cu	Fe	Hf	Li	...	Zr	density	hv	yield_strength	young_modulus	Phase
0.333	0	0	0.333	0	0	...	0	8.5	125	204	207	0
0.333	0	0	0.333	0	0	...	0	8.5	125	209	207	0
0.333	0	0	0.333	0	0	...	0	8.5		211	207	0
0.307	0	0	0.307	0	0	...	0	7.7	149	196	194	1
0.285	0	0	0.285	0	0	...	0	7.1	287	476		1
0.266	0	0	0.266	0	0	...	0	6.6	570	1301		2

Table 4 – Entries from Dataset 1 encoded for training - 26 training features in total

Encoding phase and composition enables the model to interpret the atomic structure and phase of HEAs. Additionally, all other continuous fields are designed to be flexible, the model can accept any continuous numerical data as input features or targets. Allowing the model framework to support broader experimentation for both classification and regression setups.

4.3 Workflow of the application

The model framework will provide a streamlined workflow for processing HEA compositional datasets. It parses strings into numeric features, and can calculate thermodynamic descriptors, to enrich the feature space.

Three candidate models architectures (MLP, XGB, RF – discussed in section 4.4) are available to be used as the model architecture. To enhance model performance, an optional feature selection and hyperparameter optimisation can be run.

Model validation is conducted using a 70/30 train-test split, with 70% of the data used for training and 30% reserved for testing. The model to learns patterns between input features and target values on the training set, while the test set evaluates its performance on unseen data. Model performance is measured using accuracy for classification and Root Mean Squared Error (RMSE) for regression tasks. Reliable performance on previously unobserved inputs is critical for real-world usage, making this an effective approach to model validation.

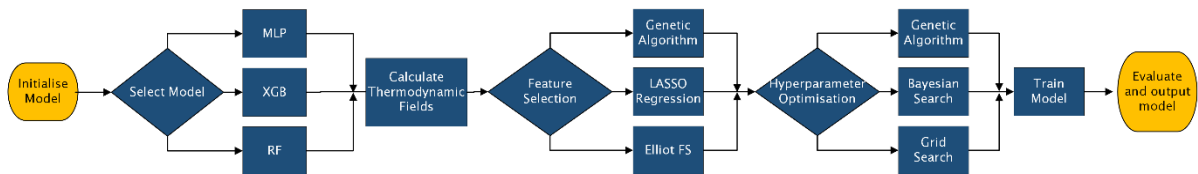


Figure 4 - Flowchart of Model design process

4.4 Machine Learning Model Selection

Three models have been selected to provide a diverse set of approaches for both classification and regression tasks. Each has demonstrated proven effectiveness in classification [20], while also offering distinct advantages for regression tasks—which is necessary for use in the CEP algorithm to fill missing dataset entries.

Random Forest

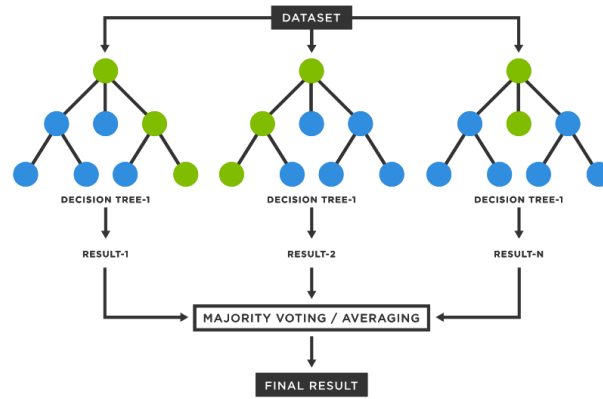


Figure 5 - Visualisation of a random forest model - multiple decision trees vote for classification or average for regression tasks [29]

Random Forest is selected due to its ability to model non-linear relationships and handle noisy data which makes it highly effective for both classification and regression tasks [30].

XGBoost

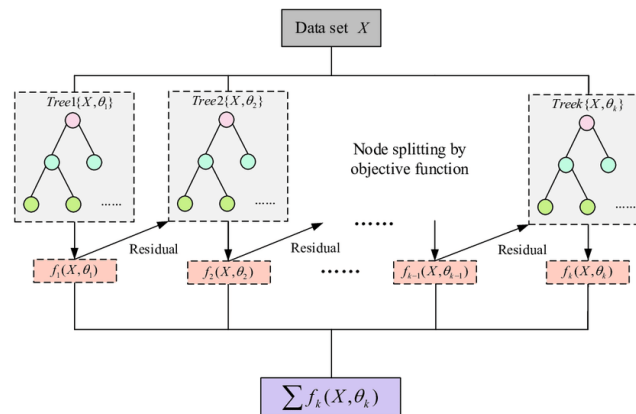


Figure 6 - Visualisation of an XGBoost model, sequential decision trees where each iteration receives on the prior iteration's residuals [31]

XGBoost offers fine control over hyperparameters such as learning rate, tree depth, and sub-sampling. It distinguishes classes in classification tasks and combines tree outputs into precise continuous predictions for regression, making it highly suitable for classification tasks alongside CEP regression tasks that require accurate estimation of continuous values [31].

Multilayer Perceptron

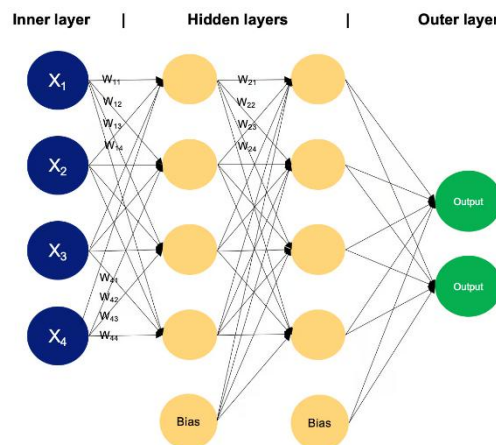


Figure 7 - Multilayer perceptron Architecture, each layer applies a weight and bias to the prior layers outputs to come to a prediction [32]

A Multilayer Perceptron (MLP) is a feed-forward neural network. This model's adaptability makes it effective across both regression and classification tasks [32].

4.4.1 Advantage of using multiple architectures

Although all three models excel at classification, they differ fundamentally in their learning approaches. Random Forest averages predictions across multiple trees for robustness [29]; XGBoost corrects errors sequentially using gradient descent with strong regularisation [31]; and MLPs use deep neural networks to model complex, non-linear patterns [32]. For the CEP algorithm, where accurate regression is critical for imputing missing data, each model offers unique advantages: Random Forest and XGBoost are noise-tolerant and stable, while MLPs provide high flexibility and expressive power. This diversity enables effective experimentation to search for the model architecture that provides the best classification performance.

4.5 Designing the Model Class

The Model Class offers a unified interface for building, evaluating, and tuning machine learning models with minimal coding. It loads data encodes phase and composition, offering multiple model architectures, feature selection, and hyperparameter optimisation, enabling fast, reproducible workflows.

It loads numerical data from CSVs, processes composition and phase data into machine-readable form, and adds thermodynamic features if specified. It then cleans and structures the data based on user-defined inputs and targets.

The class supports feature selection and hyperparameter optimisation strategies. With feature selection finding an optimal feature set and replacing the defined input features with the optimised values before training. Hyperparameter optimisation finds the optimal parameters and reinitialises the model with the new parameter set. This framework supports both classification and regression, making it ideal for the task of phase prediction in addition to application for imputation of empty fields within the CEP algorithm.

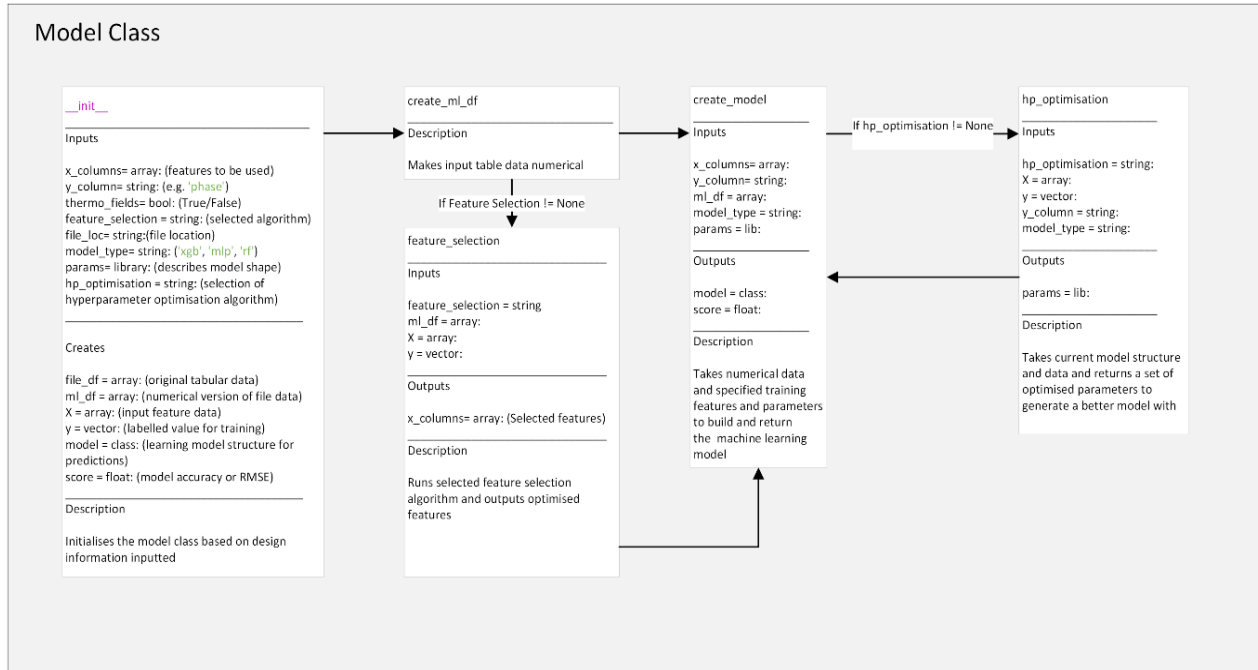


Figure 8 – Diagram showing the internal structure and function of the Model class

The Model lass enables rapid prototyping and diverse model development, integrating the model, dataset, and metadata into a single instance. This integrated approach offers a robust yet accessible tool for designing, testing, and comparing machine learning models on any numerical dataset available.

Pseudocode for the Model Class can be found in the [appendix](#).

4.6 Thermodynamic fields

The inclusion of additional thermodynamic fields expands the feature set with values that correlate with phase formation in HEAs [23]. These features influence the stability of crystal structures and help predict the phase of HEAs more accurately.

X. Chang et al. identified the following 5 properties as key values that impact phase in HEAs [23]. By combining the encoded atomic data with a periodic table database, the Model class can use the following equations in Table 5 to calculate these values.

Property	Equation
Mixing Entropy (ΔS_{mix})	$\Delta S_{mix} = -R \sum_{i=1}^n c_i \ln c_i$
Mixing Enthalpy (ΔH_{mix})	$\Delta H_{mix} = \sum_{i=1}^n \sum_{j \neq i} \Omega_{ij} c_i c_j, \quad \Omega = \frac{T_m \Delta S_{mix}}{ \Delta H_{mix} }$

Valence Electron Concentration (VEC)	$VEC = \sum_{i=1}^n c_i VEC_i$
Electronegativity difference ($\Delta\chi$)	$\Delta\chi = \sqrt{\sum_{i=1}^n c_i (\chi_i - \chi_{avg})^2}$, $\chi_{avg} = \sum_{i=1}^n \chi_i c_i$
Atomic size difference (δ)	$\delta = \sqrt{\sum_{i=1}^n c_i \left(1 - \frac{r_i}{\bar{r}}\right)^2}$, $\bar{r} = \sum_{i=1}^n r_i c_i$

Table 5 – Breakdown of thermodynamic formulas to be used as additional fields

These equations will make use of a periodic table provided by GoodmanSciences [33], which contains elemental data. This is used alongside atomic percentages for each element that are readily available with the training data, as seen in Table 4. These new calculated fields are appended to the training data as additional feature columns.

4.7 Optimisation Methods

This section outlines the optimisation strategies used to improve model performance. Feature selection and hyperparameter optimisation have been identified as effective model optimisation algorithms to improve the classification performance of the model [19].

Given the large dataset size (~90,000 entries), scalability is critical. Algorithms must process high data volumes efficiently without excessive computational cost—to manage this, algorithmic complexity is considered using Big O notation [34]. This provides a theoretical basis for comparing the scalability of each optimisation algorithm.

For completeness, pseudocode for algorithms not detailed in the main body is available in the [appendix](#).

4.7.1 Feature Selection

Feature selection identifies the most relevant features in a dataset. High-dimensionality and redundant features can reduce model performance by introducing noise and overfitting [5]. This project uses three feature selection methods:

- **LASSO Regression:** A regularisation technique that shrinks coefficients to zero for feature selection.
- **Genetic Algorithm:** An evolutionary approach selecting features based on their impact on model performance.
- **Elliot Feature Selection:** A custom algorithm based on statistical significance of feature importances.

4.7.1.1 LASSO Regression

LASSO (Least Absolute Shrinkage and Selection Operator) is a linear regression method that enhances model simplicity and prevents overfitting by applying an L_1 penalty to the loss function [35].

$$\lambda \sum_{j=1}^p |\beta_j|$$

Equation 5 - L_1 Regularisation term formula for p features [35]

This regularisation method works by shrinking some coefficients to zero during training, effectively removing redundant features. The weightings are coefficients applied to each feature in Table 4, with nonzero predictors being features which are retained.

Time complexity

LASSO regression scales linearly with dataset size, meaning computation time grows proportionally as the number of samples n increases. This makes it efficient for larger datasets.

The total time also depends on the number of features (p), since each feature's weight must be updated during training, giving an overall scaling of $O(n \cdot p)$. Additionally, the convergence tolerance (ϵ) affects how many iterations are needed, adding a $\log(1/\epsilon)$ factor. A larger regularisation parameter (λ) can speed training by zeroing out more coefficients early.

$$O(n \cdot p \cdot \log \frac{1}{\epsilon})$$

Equation 6 – Time complexity of LASSO regression

Application in context

LASSO is especially effective due to its built-in feature selection, helping reduce model complexity and overfitting. It is well-suited for high-dimensional datasets where many features may be irrelevant, making it a strong candidate for improving generalisation performance in the algorithm.

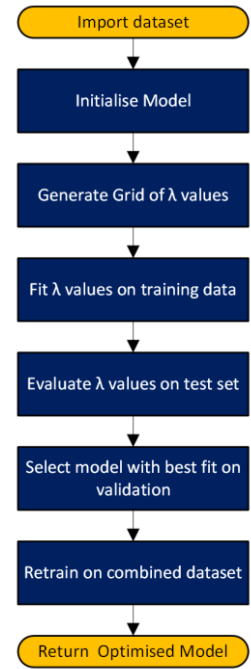


Figure 9 – Flowchart of LASSO Regression Process

4.7.1.2 Genetic algorithms

Genetic Algorithms (GAs) are optimisation techniques inspired by natural selection, using cycles of selection, crossover, and mutation to evolve solutions, demonstrated in Figure 10 [24]. In this context, each individual in the population is a binary vector, where each bit represents whether a feature is included (1) or excluded (0). The algorithm iteratively refines these vectors, attempting to find an optimal subset of features, returning a final vector representing the selected features.

Time Complexity

The time complexity of the algorithm scales approximately linearly ($O(n \log n)$) for a population n . It is approximate as the algorithm doesn't retrain on all data points every generation.

For each generation, the fitness evaluation uses a random forest, with time complexity of $O(n \log n)$. If the population has n individuals and the algorithm runs for G generations, with an initial population size P , the rough complexity is:

$$O(P \cdot G \cdot n \log n)$$

Equation 7 – Time complexity of GA for feature selection

Use of larger populations and more generations will improve search coverage but comes with a substantial increase in computational cost.

Application in context

The genetic algorithm is particularly useful in high-dimensional datasets, where brute-force evaluation of all feature combinations is impractical. However, the time complexity is linear with dataset size and time costs when expanding the population size and number of generations can have a large impact on runtime, making it more computationally expensive compared to a method like LASSO Regression.

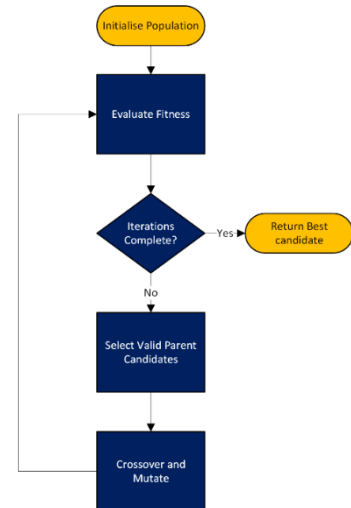


Figure 10 - Flowchart of a Genetic Algorithm's evolutionary process

4.7.1.3 Elliot Feature Selection

Elliot Feature Selection (Elliot FS) is a custom permutation-based technique for identifying statistically significant features by comparing their real importance against an uncorrelated distribution of importance. The algorithm begins by shuffling the feature values to break the correlation with the target. For a specified number of iterations, the algorithm evaluates and re-shuffles the dataset to create a null distribution for feature importance. It then calculates the real feature importance with the unshuffled dataset and compares it with the null distribution.

A feature is retained if its true importance exceeds the mean of the null distribution by a specified number of standard deviations - seen below in Figure 12. If no features meet this threshold, a fail-safe mechanism selects the most important feature.

Time complexity

The algorithm scales with the number of features (p), samples (n), and iterations (n_{iter}). In each iteration, it shuffles features which has time complexity $O(p \cdot n)$ and trains a Random Forest ($O(T \cdot d \cdot \log n)$), where T is the number of trees. This process repeats n_{iter} times, making the total time complexity:

$$O(n_{iter} \cdot p(n + T \cdot \log n))$$

Equation 8 - Time complexity of Elliot FS

Overall time complexity is linear with sample size n for shuffling. Shuffle time will be small compared to evaluation, making the model scale logarithmically. Therefore, this method is as efficient if not more efficient compared to LASSO regression.

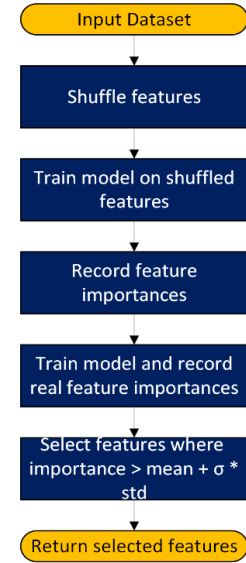


Figure 11 – Flowchart of the Elliot FS algorithm

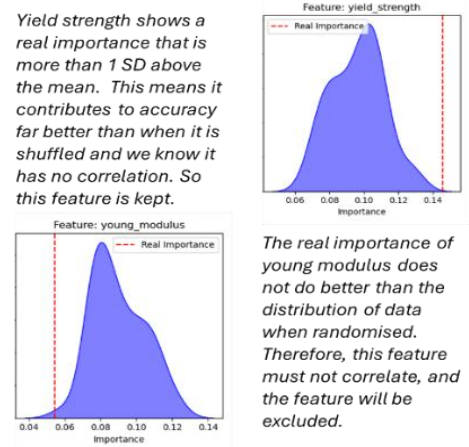


Figure 12 – Null distributions of importance for yield strength and young modulus compared to real importance.

Pseudocode of Elliot Feature Selection

Pseudocode of the Elliot FS algorithm is provided below in Figure 13.

```
1. Define class ElliotFeatureSelection with inputs:
   - n_iter: number of iterations
   - std_threshold: selection threshold (in std deviations)
   - random_state: for reproducibility

2. Initialize:
   - selected_features ← empty list
   - feature_distributions ← empty dictionary
   - real_importances ← empty dictionary

3. Function fit(X, y):
4.   Set random seed
5.   Initialize feature_importances ← dictionary of lists for each column in X
6.   FOR i FROM 1 TO n_iter DO:
7.     X_shuffled ← copy of X
8.     FOR each column IN X:
9.       Shuffle values in the column of X_shuffled
10.    Train RandomForestClassifier on X_shuffled, y
11.    FOR each column and importance:
12.      Append importance to feature_importances[column]
13.    END FOR
14.  feature_distributions ← feature_importances
15.  Compute importance_means ← mean of each column's importances
16.  Compute importance_stds ← std deviation of each column's importance
17.  Train RandomForestClassifier on real data X, y
18.  real_importances ← feature_importance for each column in X
19.  selected_features ← list of columns WHERE:
    real_importance > (mean + std_threshold × std)
20.  IF selected_features is empty:
21.    selected_features ← column with highest real_importance
22.  RETURN self

23. Function plot_feature_distribution(feature=None):
24.  IF feature is provided AND exists in feature_distributions:
25.    features_to_plot ← [feature]
26.  ELSE:
27.    features_to_plot ← all keys in feature_distributions
28.  FOR each feature IN features_to_plot:
29.    Plot kernel density of importance distribution
30.    Add vertical line for real_importance
31.    Add legend and axis label
32.  Display the plots

33. Function transform(X):
34.  RETURN X[selected_features]

35. Function fit_transform(X, y):
36.  CALL fit(X, y)
37.  RETURN transform(X)
```

Figure 13 - Pseudocode of the Elliot FS algorithm

Application in context

Elliot FS selects features by comparing real feature importances to those from shuffled data, keeping only statistically significant ones. It captures nonlinear relationships and is robust to noise. Additionally, it is deterministic and more interpretable. It scales partially linearly and partially logarithmically relative to sample size, making it more efficient than Genetic algorithm, and equivalent or better compared to LASSO Regression.

4.7.2 Hyperparameter Optimisation

Hyperparameter optimisation is essential for fine-tuning machine learning models, aiming to find the best combination of hyperparameters that significantly improve model performance [25]. Unlike model parameters, which are learned during training, hyperparameters are set

before training begins and can greatly influence model accuracy, convergence speed, and robustness [25].

This section discusses key methodologies for hyperparameter tuning, including both traditional and advanced techniques to explore the hyperparameter space. The following optimisation algorithms will be used:

- **Bayesian Search:** Uses probabilistic models to explore the hyperparameter space efficiently, prioritising areas likely to yield better results based on prior evaluations.
- **Parallel Grid Search:** Exhaustively tests all possible combinations of specified hyperparameter values - optimised with parallelisation to speed up evaluation.
- **Genetic Algorithm:** Mimics natural selection by evolving a population of hyperparameter sets over generations to find optimal values

4.7.2.1 Bayesian Search

Bayesian Search is a surrogate model-based optimisation method used to efficiently find the global optimum of a cost function, such as validation error. It leverages a Gaussian Process (GP) to model the relationship between hyperparameters and model performance, guiding the search for optimal parameters [27]. In this context, the method is used to tune architecture parameters to maximise the cost function (Accuracy or RMSE).

Time Complexity

The time complexity of Bayesian Search depends on the sample size n and the model type being optimised:

- For Random Forest and XGBoost, training scales as $O(n \log n)$. XGBoost may be slightly more expensive due to its gradient boosting iterations.
- For MLPs, training complexity is $O(n p)$, where p is the number of parameters.

The total time complexity for k evaluations is roughly:

$$O(k \cdot T(n)) \text{ where } T(n) \in \{O(n \log n), O(n p)\}$$

Equation 9 – Time complexity of a Bayesian Search

Additionally, the GP surrogate model introduces an overhead of $O(k^2)$ due to the need for matrix operations, but this is small compared to the time spent on model training and can be considered as negligible.

Application in context

Bayesian Search is a highly effective tool for optimising models. By using a Gaussian Process to intelligently select hyperparameters, it reduces the number of evaluations required to

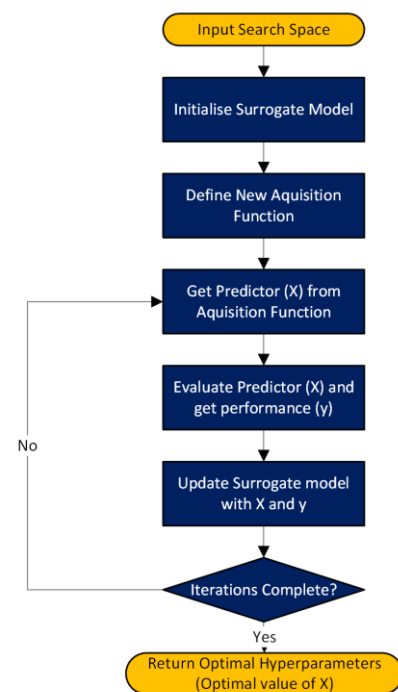


Figure 14 - Flow chart of the Bayesian Search algorithm

find an optimal configuration, making it especially valuable for large datasets or computationally expensive models. However, it can scale in a greater than linear fashion, and this results in high computational cost for larger datasets.

4.7.2.2 Parallel Grid Search

Grid Search is a brute-force method for hyperparameter tuning that evaluates all possible combinations of hyperparameters. It is simple but can be computationally expensive, especially with large datasets or a broad search space [36]. The method has been improved by integration of multiprocessing, to parallelise evaluation steps.

Time Complexity

The time complexity depends on the number of hyperparameter combinations and the dataset size n :

- MLP: $O(n \cdot p)$, where p is the number of parameters.
- XGBoost/Random Forest: $O(n \log n)$.

For a hyperparameter space with h_k possible values for each parameter, possible values for each parameter, the total number of combinations is the product of these values $\prod_{i=1}^k h_i$. Therefore, the total time complexity is:

$$O\left(\prod_{i=1}^k h_i \cdot T(n)\right) \text{ where } T(n) \in \{O(n \log n), O(n p)\}$$

Equation 10 - Time complexity of a grid search without multiprocessing

This high time complexity is reduced with multiprocessing, allowing operations to be evaluated in parallel. For q parallel processes, the total time complexity becomes the following equation.

$$O\left(\frac{\prod_{i=1}^k h_i \cdot T(n)}{q}\right) \text{ where } T(n) \in \{O(n \log n), O(n p)\}$$

Equation 11 - Time complexity of a grid search with multiprocessing

This results in a more manageable runtime, especially for large hyperparameter spaces, though the overall complexity still depends on the number of combinations and the size of the dataset.

Application in context

Grid Search is a simple but computationally expensive method for hyperparameter optimisation. While it guarantees finding the best hyperparameters, its time complexity grows rapidly

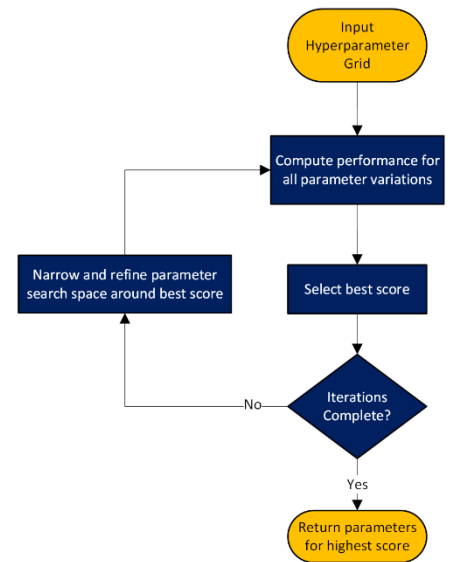


Figure 15 - Flowchart of the Grid Search Algorithm

with the number of hyperparameters and dataset size. The method's efficiency can be significantly improved through multiprocessing, which allows multiple evaluations to be run concurrently. However, Grid Search remains best suited for smaller hyperparameter spaces or when computational resources are available for parallel execution.

4.7.2.3 Genetic algorithm

A Genetic Algorithm is also used for hyperparameter optimisation, following an evolutionary cycle—evaluation, selection, crossover, and mutation—to evolve candidate solutions over generations. Unlike its use in feature selection where inputs are binary (present/absent), here the input vector consists of continuous values, representing different hyperparameters [26].

Time Complexity

The time complexity of GA scales with the dataset size primarily through the model training cost $T(n)$, repeated across many generations and candidate solutions:

- MLP: $O(n \cdot p)$ – forward/backward passes scale with data and number of parameters.
- XGBoost / Random Forest: $O(n \log n)$ – tree-building operations grow logarithmically with dataset size.

Overall, the total time complexity is:

$$O(G \cdot P \cdot T(n)) \text{ where } T(n) \in \{O(n \log n), O(n p)\}$$

Equation 12 - Time complexity of the genetic algorithm for hyperparameter optimisation

Where G represents the number of generations the algorithm runs through, P is the population of candidates evaluated in each generation, and $T(n)$ denotes the training time for each model, which depends on the underlying model architecture. The model scales linearly or linearithmically with sample size, dependent on model architecture. This is the same as all other methods, however the model can see fast growth in runtime with population size and more generations.

Application in context

The Genetic Algorithm provides an alternative to brute-force methods. Compared to Grid Search, it avoids exhaustive evaluation and explores the space more efficiently. While it is generally less sample-efficient than Bayesian Search, which uses a surrogate model (e.g., Gaussian Process) to intelligently propose candidates, GA can perform better in highly complex or noisy search spaces. GAs offer a strong balance between search depth and runtime, making them well-suited for larger or more irregular hyperparameter landscapes.

4.8 Crude Estimation of Property

The Crude Estimation of Property (CEP) algorithm is inspired by work done by Y. Zhang et al. and imputes missing values in datasets [6]. However, it does not use averaging methods. Instead, it operates by iteratively filling columns one at a time. Each iteration the algorithm evaluates a variety of model architectures in parallel, to find an optimal model with the lowest RMSE.

CEP starts by copying the dataset and identifying columns with missing values. It tests various base models (e.g., XGBoost, Multilayer Perceptron, Random Forest) on each target column, selecting the best one based on RMSE.

The chosen model is then refined through feature selection, hyperparameter tuning, and integration of thermodynamic fields, evaluating and selecting the optimisation methods which yield the lowest RMSE. Then using the optimal model to fill a column - this process is repeated until all columns are filled.

Upon completion the algorithm outputs a fully imputed dataset and a metadata stack containing model details, settings, and RMSE scores, ensuring transparency of the search process.

CEP surpasses basic methods like mean or median filling by exploiting inter-feature relationships, though it can be computationally demanding. To address this, it employs worker pools, early stopping, and shared memory to optimise resource usage.

Its modular design using the model class supports easy integration of new models and optimisation strategies, making CEP a robust, scalable solution for high-quality imputation in large, complex datasets.

Pseudocode for the CEP algorithm is provided below in Figure 17.

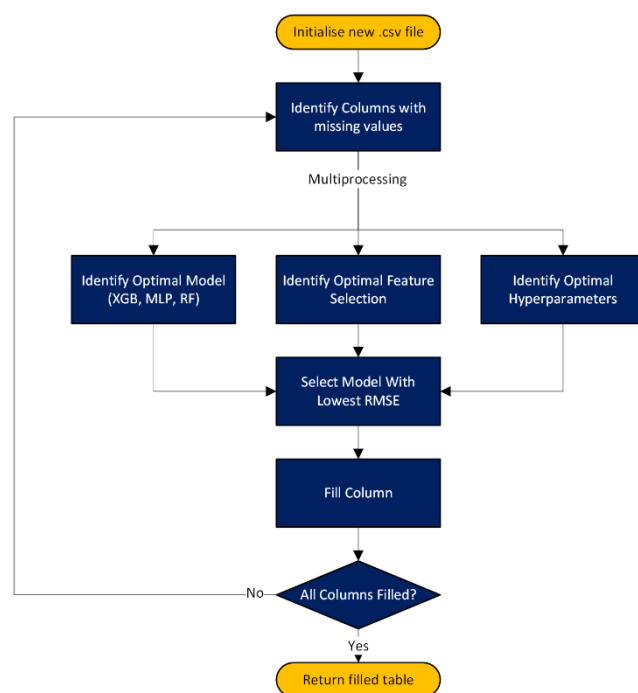


Figure 16 - Flowchart of CEP algorithm, the diverging section shows where operations are split across multiple CPU cores

Pseudocode for the CEP algorithm

```
1. FUNCTION fill_table(file_path, y_column, optional training_data_path):
2.   stack ← empty list to store fitted model results
3.   new_file_path ← copy of file_path (preserve original data)
4.   null_columns ← columns in new_file_path with missing values
5.   Remove y_column from null_columns
6.   non_empty_columns ← columns in new_file_path with no missing values
7.   WHILE null_columns is not empty:
8.     IF training_data_path is provided:
9.       best_model ← get_best_model(training_data_path, null_columns, non_empty_columns)
10.    ELSE:
11.      best_model ← get_best_model(new_file_path, null_columns, non_empty_columns)
12.      df_filled ← fill_missing_values(new_file_path, best_model)
13.      Save df_filled to new_file_path
14.      filled_col ← best_model.y_column
15.      Remove filled_col from null_columns
16.      Append filled_col to non_empty_columns
17.      Append best_model to stack
18.   RETURN stack

19. FUNCTION get_best_model(file_path, null_columns, non_empty_columns):
20.   x_columns ← non_empty_columns
21.   y_columns ← null_columns excluding 'phase'
22.   model_types ← ['xgb', 'mlp', 'rf']
23.   model_configs ← []
24.   FOR y_column IN y_columns:
25.     FOR model_type IN model_types:
26.       Append (model_type, x_columns, y_column, basic settings) to model_configs
27.   Start multiprocessing queue and run model training jobs
28.   Collect results from queue into results list
29.   Join and terminate all processes
30.   best_model ← model with lowest RMSE in results
31.   Refine best_model with hyperparameter and feature selection:
32.     FOR all combinations of:
33.       feature_selection ∈ ['genetic', 'elliott', 'lasso', None]
34.       hp_optimisation ∈ ['bayesian', 'grid', 'genetic', None]
35.       thermo_fields ∈ [True, False]
36.       Append refined config to model_configs
37.   Start multiprocessing queue and evaluate refined configs
38.   Collect and join results
39.   best_model ← model with lowest RMSE among refined
40.   RETURN best_model

41. FUNCTION fill_missing_values(file_path, model_result):
42.   model ← model_result.model
43.   target_column ← model_result.y_column
44.   file_df ← read file_path
45.   model.file_df ← file_df // Rebind model to new data
46.   ml_df ← model.create_ml_df()
47.   train_df ← ml_df[model.x_columns + [target_column]]
48.   missing_mask ← train_df[target_column].is null
49.   IF no missing values:
50.     RETURN original file_df
51.   X_missing ← rows from train_df where target_column is null (excluding target_column)
52.   X_missing_np ← convert X_missing to NumPy array
53.   predicted_values ← []
54.   FOR i from 0 to len(X_missing_np) step batch_size:
55.     batch ← X_missing_np[i:i+batch_size]
56.     predicted_values += model.predict(batch)
57.   Insert predicted_values back into missing rows of train_df
58.   file_df[target_column] ← updated column from train_df
59.   RETURN file_df
```

Figure 17 - Pseudocode of the CEP algorithm

5. Results, Analysis and Discussion

This section evaluates the performance of model architectures and optimisation methods, with the aim of identifying an optimal configuration. The selected model is then subjected to a detailed analysis to evaluate the model's accuracy, stability, robustness, validation, and calibration, followed by a discussion on its potential applicability in engineering contexts.

5.1 Baseline Model accuracy

Baseline models were evaluated without enhancements to establish a reference for later optimisation. Classification accuracy and regression performance were assessed, alongside model stability through repeated randomisation of training and validation sets.

The aim was to select the best architecture for HEA phase classification. Mean accuracies are given below alongside a Confusion Matrix in Table 6.

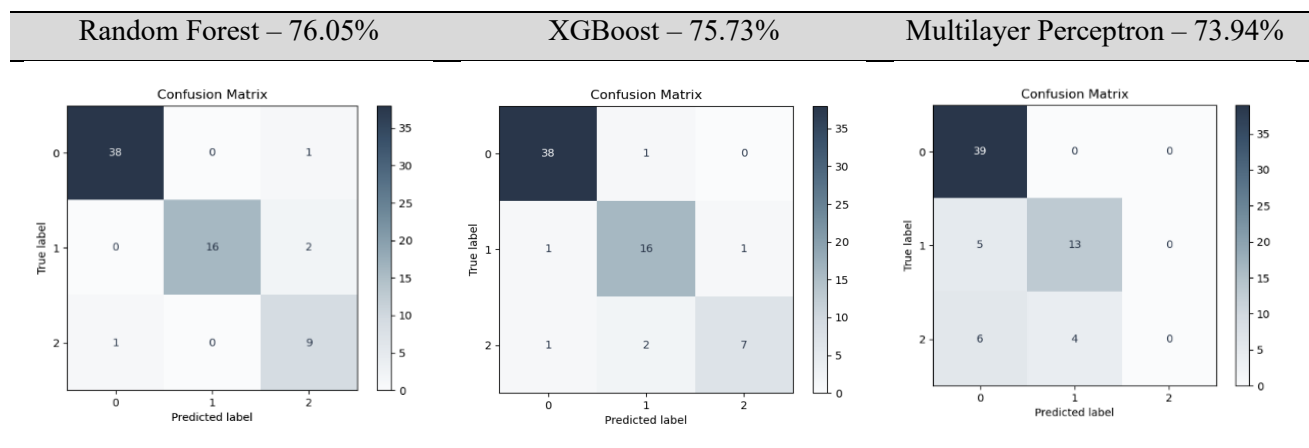


Table 6 – Baseline model accuracies when predicting phase (0: FCC, 1: BCC or 2: Other)

Among the baseline models, Random Forest (RF) achieved the highest mean accuracy at 76.05%, closely followed by XGBoost (XGB) at 75.74%. The Multi-Layer Perceptron (MLP) performed worse, likely due to difficulties with the high number of input features. Tree-based models like RF and XGB are more robust to high-dimensional spaces [31].

Beyond accuracy, model stability was assessed by randomising train-test splits and analysing the resulting performance distributions to evaluate consistency. The distribution of accuracy can be seen in Table 7 and Figure 18.

Model Type	Mean Accuracy (%)	Standard deviation (%)
Random Forest	76.05%	8.08%
XGBoost	75.73%	4.85%
Multilayer Perceptron	73.94%	6.56%

Table 7 – mean and standard deviation of accuracy for baseline models

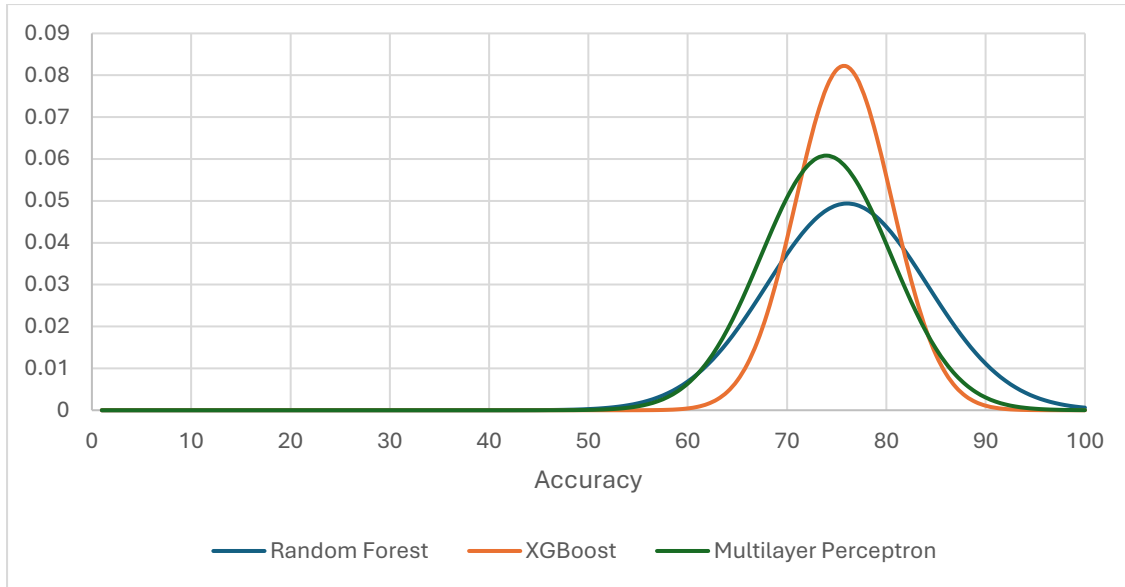


Figure 18 – Distribution of accuracy for baseline models

XGBoost demonstrated higher stability in comparison to the random forest and multilayer perceptron (~2% lower standard deviation than MLP and ~3% higher than RF).

In addition to classification, the models were evaluated for regression performance using RMSE to ensure they are suitable for the CEP algorithm, which requires imputation of continuous variables.

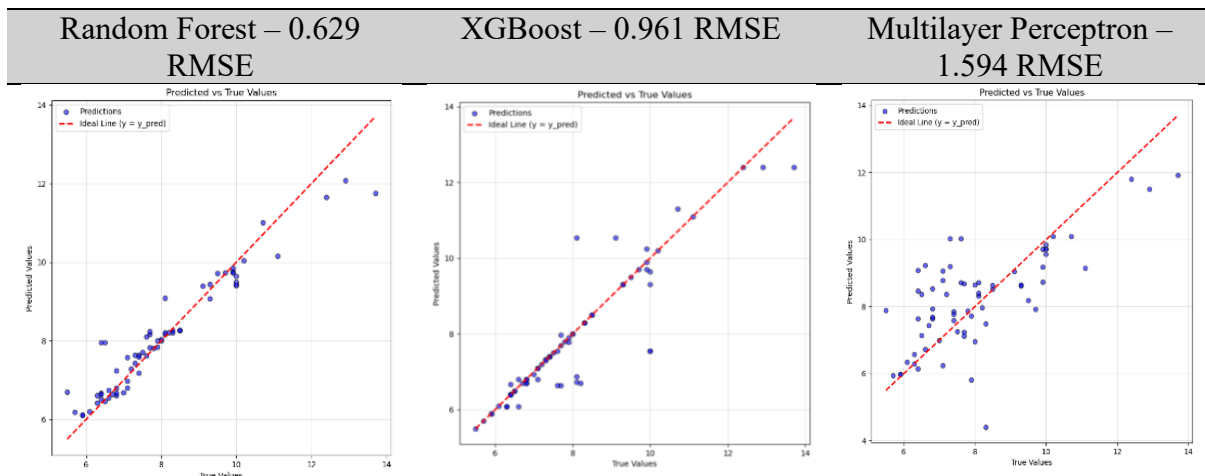


Table 8 – mean RMSE for baseline models alongside plots of predicted/true values

All models achieved low RMSEs for predicting Young’s Modulus (Table 8), demonstrating these models are viable for use in the CEP algorithm.

XGBoost is noticeably more stable than random forest and multilayer perceptron in classification, despite having lower accuracy than random forest (~0.5%), this loss is justifiable when considering XGBoost’s higher stability, which is critical to producing a reliable model. Based on these results, XGBoost is chosen as the model for use in phase prediction.

5.2 Assessment of algorithms

Optimisation algorithms are evaluated based on four criteria:

- **Predictive Accuracy** (percentage improvement in mean accuracy),
- **Model Stability** (standard deviation of accuracy and RMSE),
- **Runtime Efficiency** (runtime of training and evaluation)
- **Viability for CEP** (improvement in RMSE).

Each algorithm is tested over 100 randomised train-test splits, recording both mean and standard deviation of Accuracy and RMSE (dependent on classification or regression task). Although RMSE does not impact the primary task of phase classification, it is critical for the CEP algorithm. Therefore, RMSE reduction should also be carefully analysed. All comparisons will be compared to the unoptimised XGB model.

Finally, a hypothesis test at the 2.5% significance level assesses whether accuracy gains are statistically significant, and the best-performing optimisation method will be selected for use in the final phase prediction model.

5.2.1 Thermodynamic Fields

Thermodynamic fields—mixing entropy, enthalpy, valence electron concentration, electronegativity difference, and atomic size difference—were derived from composition data and a periodic table database [33] to enrich the model's input space. Their inclusion led to a 3.68% increase in mean classification accuracy (from 75.73% to 79.41%) and a reduction in standard deviation of accuracy (from 4.857% to 4.412%), improving both performance and stability. Although model runtime increased $\sim 10\times$, the absolute runtime remained low at only 1.032 s. Since time complexity is proportional to dataset size this is not an issue, and the method is scalable for larger datasets.

	Unoptimised	With Thermodynamic Fields
Mean Accuracy (%)	75.73%	79.41%
Change in Accuracy (%)	N/A	+3.68%
Standard Deviation in Accuracy (%)	4.857%	4.412%
Change in Standard Deviation of Accuracy (%)	N/A	- 0.446%
Model Runtime (s)	0.11	1.032

Table 9 – Improvement in classification with addition of thermodynamic fields

Addition of thermodynamic fields not only delivers higher mean accuracy but also exhibits lower standard deviation in accuracy, suggesting more stable performance.

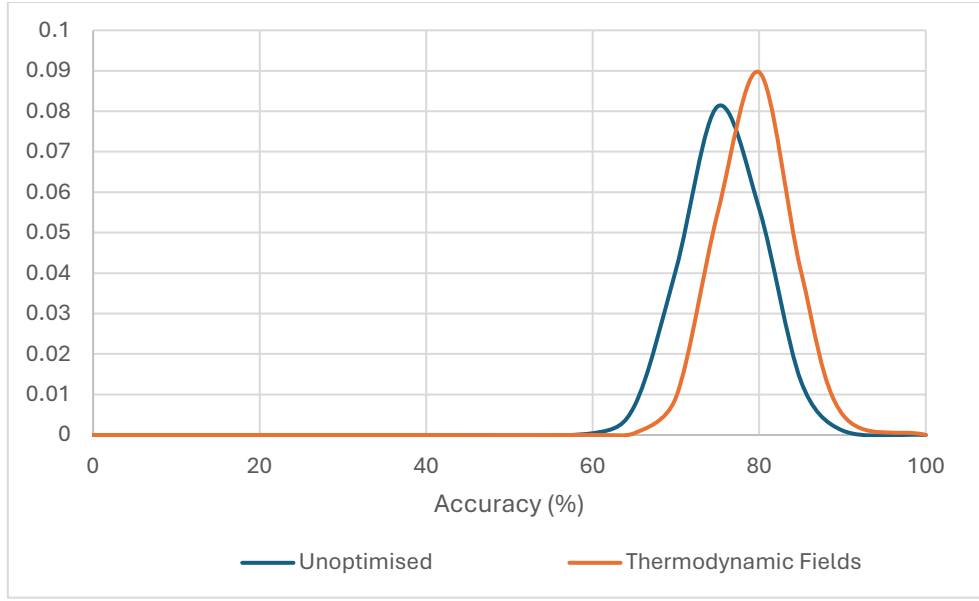


Figure 19 - Distribution of accuracy with and without Thermodynamic Fields

Regression Performance

Models were also evaluated in regression, using distribution of RMSE in the prediction of Young's Modulus as a performance metric.

Addition of thermodynamic fields resulted in a reduction in mean RMSE of -0.013. Furthermore, the standard deviation in RMSE was reduced by -0.137, suggesting that the addition of thermodynamic fields produces more stable regression results. Confirming the addition of thermodynamic fields is suitable for use within the CEP algorithm.

	Unoptimised	Thermodynamic Fields
Mean RMSE	0.725	0.712
Change in RMSE	N/A	-0.013
Standard deviation of RMSE	0.258	0.121
Change in Standard Deviation of RMSE	N/A	-0.137
Model Runtime (s)	0.11	1.032

Table 10 – Regression performance of thermodynamic fields compared to an unoptimised XGBoost model

Statistical significance of Thermodynamic Fields

A hypothesis test is carried out to decide whether calculation of thermodynamic fields produce a significant improvement in mean accuracy of a model. The mean accuracy of an unoptimised model is 75.3%, with a standard deviation of 4.857. Mean accuracy with thermodynamic fields was 79.41 across 100 samples. The hypothesis test was done at a 2.5% significance level.

$$X \sim N\left(\mu, \frac{4.857^2}{100}\right)$$

$$H_0: \mu = 75.73$$

$$H_1: \mu > 75.73$$

$$P(X = 79.41) = 3.69 \times 10^{-5}$$

$$2.591 \times 10^{-13} \ll 0.025$$

The new hypothesis is accepted, and it is concluded that addition of thermodynamic fields produces an improvement in model accuracy.

Addition of thermodynamic fields can be accepted as an effective method for improving model accuracy and RMSE. Additionally, reduction in standard deviation demonstrates an improvement in model stability.

5.2.2 Feature Selection

Feature selection techniques were employed to eliminate redundant or non-informative features, reducing overfitting and improving model generalisation, stability, and efficiency. Three methods were tested:

- Genetic Algorithm
- LASSO Regression
- Elliot Feature Selection

Each technique led to measurable changes in classification accuracy, stability, and runtime, detailed in the following analysis.

Classification Performance

	Unoptimised	Elliot FS	Genetic	LASSO
Mean Accuracy (%)	75.73%	77.90%	77.27%	76.42%
Change in Accuracy (%)	N/A	+2.17%	+1.54%	+0.69%
Standard Deviation in Accuracy (%)	4.857%	3.05%	4.69%	3.20%
Change in Standard Deviation of Accuracy (%)	N/A	-1.81%	-0.172%	-1.656%
Model Runtime (s)	0.11	2.80	6.15	2.81

Table 11 – Classification performance of Feature Selection algorithms compared to an unoptimised XGBoost model

All feature selection methods improved classification accuracy and reduced standard deviation, confirming that removing redundant features improves model stability, leading to more consistent classification accuracies. Elliot FS showed the best performance, with a +2.17% increase in mean accuracy and the lowest standard deviation (3.05%). Its runtime (2.80 s) was also efficient, similar to LASSO and much faster than the Genetic Algorithm (6.15 s). With time complexity scaling linearly or better with dataset size, Elliot FS is clearly the highest performing in terms of classification performance.

The shift in accuracy distributions across methods is illustrated in the figure below.

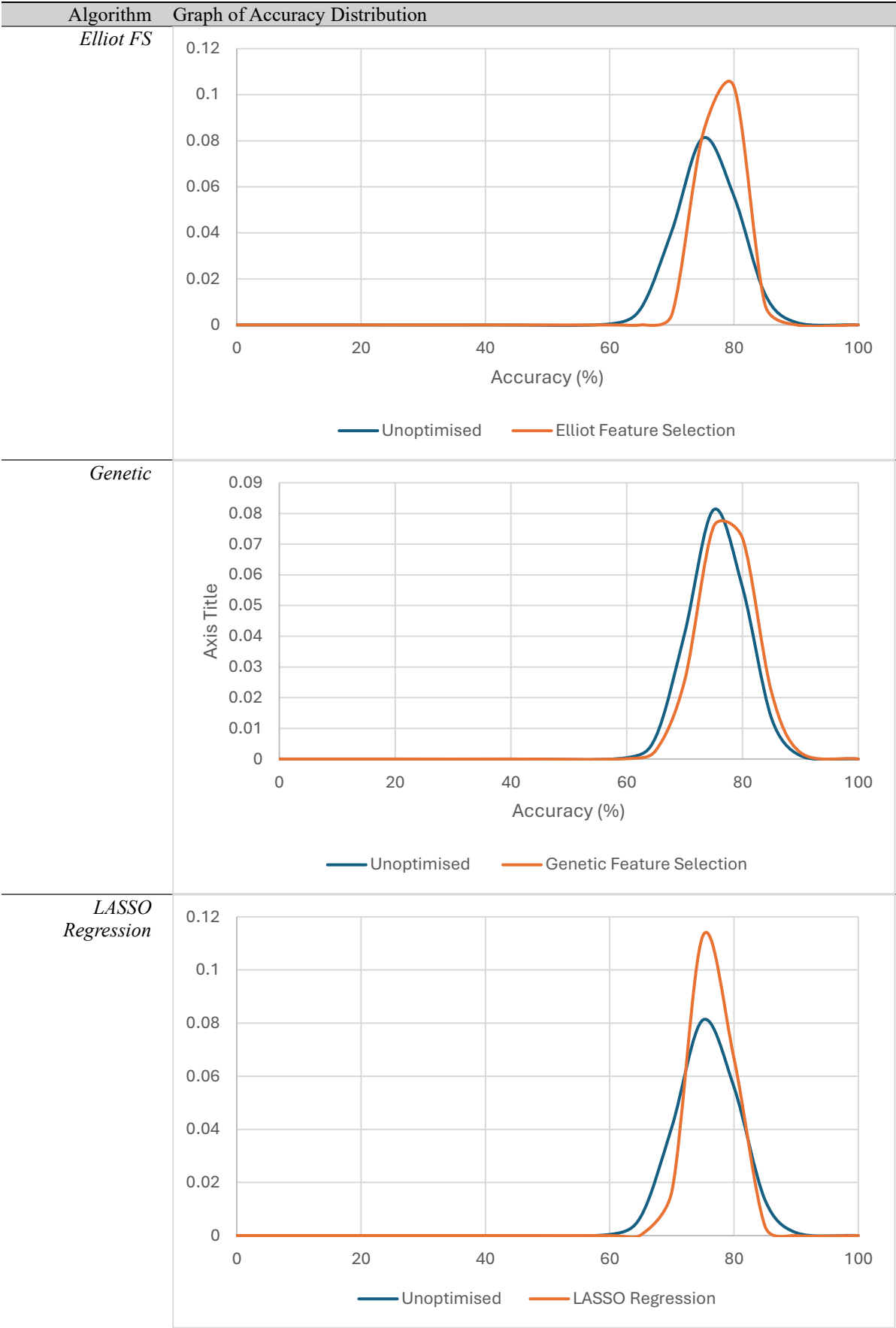


Figure 20 – Distribution of model performance compared to unoptimised XGBoost Model

Regression Performance

	Unoptimised	Elliot FS	Genetic	LASSO
Mean RMSE	0.725	0.536	0.490	0.586
Change in RMSE	N/A	-0.190	-0.236	-0.140
Standard deviation of RMSE	0.258	0.129	0.128	0.257
Change in Standard Deviation of RMSE	N/A	-0.129	-0.130	-0.001
Model Runtime (s)	0.11	2.80	6.15	2.81

Table 12 - Regression performance of Feature Selection algorithms compared to unoptimised XGBoost model

All methods improved mean RMSE predicting Young's Modulus, the Genetic Algorithm achieved the lowest RMSE (0.490), with Elliot FS close behind (0.536), and LASSO showing the least improvement (0.586). Elliot FS and Genetic methods showed reduction in standard deviation, leading to higher precision in regression. These results confirm that feature selection improves regression performance, validating its use within the CEP framework.

Elliot FS is selected as the feature selection method for use in the final model, since it showed the greatest improvement in accuracy and stability.

Statistical Significance of Accuracy Improvement

A hypothesis test was conducted to assess if Elliot FS improves model accuracy. An unoptimised XGBoost model has a mean accuracy of 75.73% (std dev = 4.857). After applying Elliot FS, the model achieved a mean accuracy of 77.90% across 100 randomised samples. The hypothesis test was performed at a 2.5% significance level:

$$X \sim N\left(\mu, \frac{4.857^2}{100}\right)$$

$$H_0: \mu = 75.73$$

$$H_1: \mu > 75.73$$

$$P(X = 77.90) = 3.69 \times 10^{-5}$$

$$3.69 \times 10^{-5} \ll 0.025$$

Since the p-value is less than the significance level, the null hypothesis is rejected, confirming that the accuracy improvement is statistically significant.

5.2.3 Hyperparameter Optimisation

Hyperparameter optimisation techniques aim to identify the model parameter configuration that minimises the cost function for a given model. Three optimisation strategies were evaluated:

- Genetic Algorithm
- Grid Search
- Bayesian Optimisation

Accuracy Performance

	Unoptimised	Genetic	Bayesian Search	Grid Search
Mean Accuracy (%)	75.73%	80.51%	83.08%	79.41%
Change in Accuracy (%)	N/A	+4.78%	+7.35%	+3.67%
Standard Deviation in Accuracy (%)	4.857%	6.57%	2.43%	4.41%
Change in Standard Deviation of Accuracy (%)	N/A	+ 1.71%	- 2.42%	- 0.4%
Model Runtime (s)	0.11			

Table 13 - Classification performance of Hyperparameter Optimisation algorithms compared to an unoptimised XGBoost model

All optimisation methods improved classification accuracy. Bayesian Optimisation achieved the highest mean accuracy (83.08%) with the greatest reduction in accuracy variance (−2.42%), demonstrating its effectiveness in converging to a point of highest accuracy.

The Genetic Algorithm demonstrated high variability in classification performance, with a rise of 1.71% in standard deviation, indicating reduced model stability. Grid Search offered marginal gains in accuracy and stability alongside shorter runtimes, demonstrating the effectiveness of the parallel processing applied.

Bayesian Optimisation yielded the most significant improvement in accuracy. Given that all approaches share a linearithmic time complexity when applied to XGBoost models, they scale similarly with dataset size. Despite increased computational cost of Bayesian Search, the decision to use this method is justified by its superior accuracy and reliability. Based on these results, Bayesian Search is selected to be used in the final phase prediction model.

Improvements in accuracy distribution can be seen below in Figure 21.

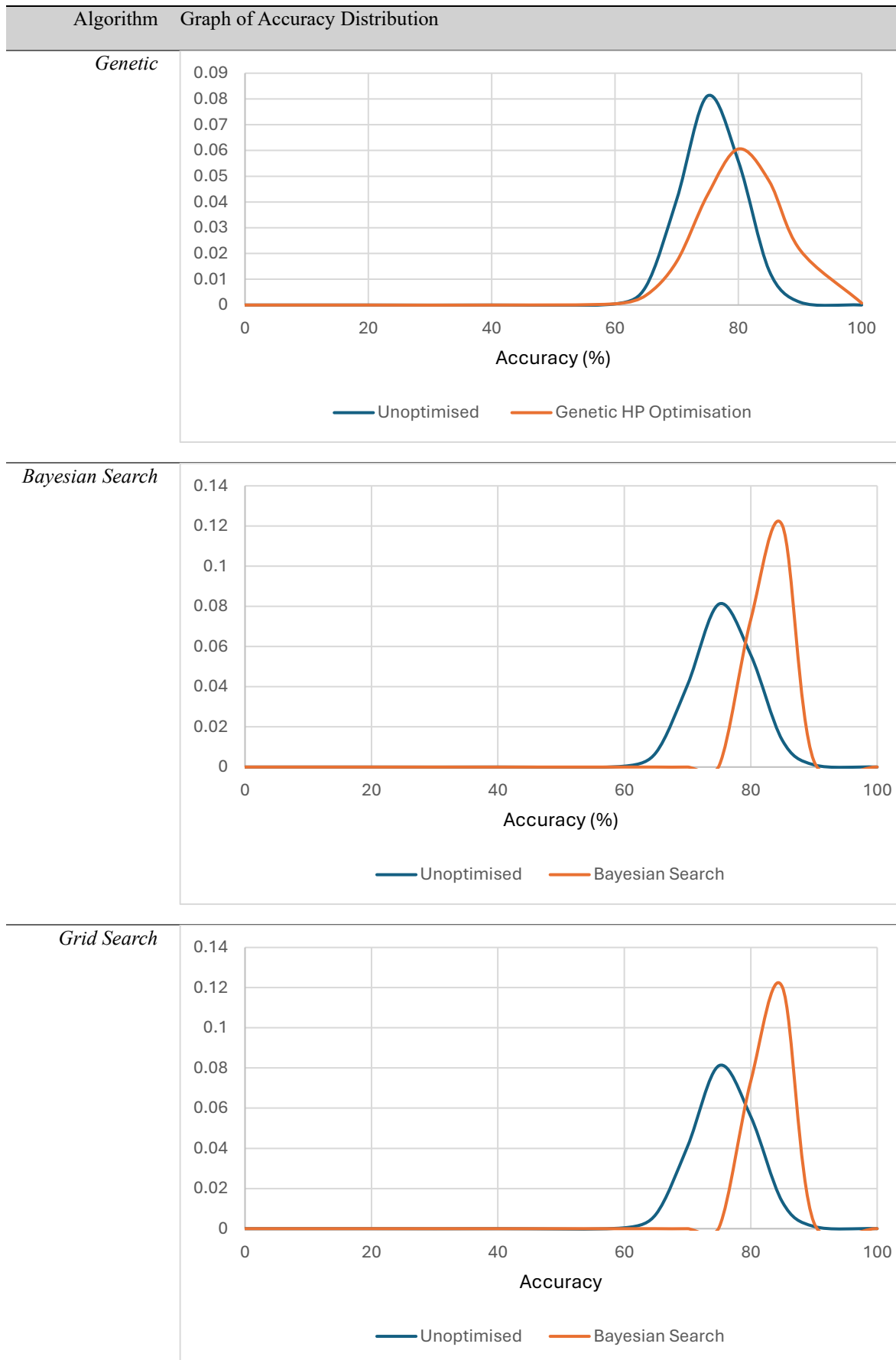


Figure 21 - Distribution of accuracy of Hyperparameter Optimisation algorithms compared to unoptimised XGBoost model

Regression Performance

	Unoptimised	Genetic	Bayesian Search	Grid Search
Mean RMSE	0.725	0.603	0.641	0.655
Change in RMSE	N/A	-0.122	-0.085	-0.071
Standard deviation of RMSE	0.258	0.109	0.130	0.104
Change in Standard Deviation of RMSE	N/A	-0.149	-0.128	-0.155
Model Runtime (s)	0.11	33.32	82.64	4.26

Table 14 - Regression performance of Hyperparameter Optimisation algorithms compared to an unoptimised XGBoost model

All optimisation methods effectively reduced the RMSE and standard deviation, indicating improved regression performance and stability. These improvements support the application of hyperparameter tuning in the CEP algorithm.

Statistical Significance of Accuracy Improvement

To validate the effectiveness of Bayesian optimisation, a statistical hypothesis test was performed. The baseline model achieved a mean accuracy of 75.73% with a standard deviation of 4.857%. After applying Bayesian optimisation, the model's mean accuracy increased to 83.08% over 100 samples. A one-tailed hypothesis test at a 2.5% significance level was conducted.

$$X \sim N\left(\mu, \frac{4.857^2}{100}\right)$$

$$H_0: \mu = 75.73$$

$$H_1: \mu > 75.73$$

$$P(X = 77.90) = 1.539 \times 10^{-50}$$

$$1.539 \times 10^{-50} \ll 0.025$$

Therefore, the null hypothesis is rejected, confirming that Bayesian optimisation significantly improved model accuracy.

Bayesian Search proved the most effective hyperparameter optimisation method for phase prediction, delivering significant improvements in accuracy and stability. Despite longer runtimes, its superior accuracy and consistency justify its selection for the final model.

5.2.4 Crude estimation of property

The Crude Estimation of Property (CEP) algorithm is the final optimisation step in the model pipeline, used to impute missing values using predictions with optimised regression models. The approach identifies the model configuration with the lowest Root Mean Squared Error

(RMSE) and uses it to fill incomplete entries. Providing additional training data with the aim of improving predictive performance.

Running the CEP algorithm on Dataset 1 (seen in Table 2) took 1342.38 seconds. Compared to 1953.80 seconds without parallelisation, this shows a 31.29% reduction in runtime, which is valuable for larger datasets.

	Unoptimised	With a CEP Filled dataset
Mean Accuracy (%)	75.73%	83.15%
Change in Accuracy (%)	N/A	+7.42%
Standard Deviation in Accuracy (%)	4.857%	2.139%
Change in Standard Deviation of Accuracy (%)	N/A	- 2.718%
Time to fill table (s)	N/A	1342.38

Table 15 – Classification performance of an XGBoost model using the original dataset compared to a dataset filled by the CEP algorithm

Use of a CEP filled dataset resulted in a growth of 7.42% for mean accuracy, alongside 2.718% reduction of standard deviation. These improvements in classification performance are the largest of all optimisation methods, reinforcing how important the quantity of training data is upon model accuracy, even if imputed data isn't completely accurate.

Statistical Significance of the CEP algorithm

A one-tailed hypothesis test with 100 independent models trained on the CEP-filled dataset confirmed statistical significance. The test rejected the null hypothesis ($\mu=75.73$) with a p-value of 1.721×10^{-51} , well below the 2.5% significance level, validating the CEP's impact on accuracy.

$$X \sim N\left(\mu, \frac{4.857^2}{100}\right)$$

$$H_0: \mu = 75.73$$

$$H_1: \mu > 75.73$$

$$P(X = 77.90) = 1.721 \times 10^{-51}$$

$$1.721 \times 10^{-51} \ll 0.025$$

Overall, the CEP algorithm significantly outperforms other optimisation methods, improving both accuracy and stability, making it a key component in final model training and preprocessing for all datasets.

5.3 Building an optimal model

This section details the development of the optimal machine learning model for phase prediction in HEAs. Optimisation methods were selected based on findings from section 5.2. The model was designed to balance high classification accuracy, low variance for stability, and computational efficiency for scalability.

5.3.1 Model architecture

The selected core predictive model is XGBoost, which demonstrated strong overall accuracy and stability across iterations. Then, thermodynamic fields will be added due to their 3.68% increase in mean accuracy, and 0.446% reduction in standard deviation.

The Crude Estimation of Property (CEP) algorithm is used to impute missing values of all datasets, increasing the number of samples available for training. In testing, use of a CEP filled dataset resulted in a 7.42% improvement in mean accuracy. It also significantly improved stability, reducing the standard deviation of accuracy by 2.718%.

For optimisation of the model, Elliot FS is the chosen feature selection method, since it had the highest improvement in accuracy (+2.17%), alongside the highest reduction in standard deviation of accuracy (-1.81%), demonstrating clear enhancement in classification accuracy and stability.

Hyperparameter tuning is done with Bayesian Optimisation, which demonstrated the largest growth in mean accuracy (+7.35%). It also reduced standard deviation ~3-5% more than grid search and genetic algorithms. Despite its longer runtimes, it has the same time complexity as other algorithms, and its poorer performance is justified due to its accuracy and stability improvements.

5.3.2 Model Training

The final model was trained on a high-quality, diverse dataset. Pre-training used the large database of 84,024 Density Functional Theory (DFT) values, allowing the model to learn general physical trends. The primary training dataset included entries from Dataset 2 and a randomised 50% split of Dataset 1, totalling 1,772 training samples. Validation was conducted using the remaining 50% of the HEA dataset, leaving 198 HEA unseen samples for testing to ensure reliable performance metrics.

The optimal model integrates XGBoost with added thermodynamic features, Elliot FS for feature refinement, Bayesian Search for hyperparameter tuning, and CEP imputed datasets. This architecture combines high predictive accuracy with stability and scalability, making it well-suited for this classification task.

5.4 Model Analysis

The optimised model was evaluated using a series of tests to assess accuracy, robustness, generalisation, and calibration.

- **Accuracy Testing:** Compare mean and standard deviation of accuracy across 100 iterations to benchmark improvements over the unoptimised model.

- **Robustness to Noise:** Add 2%, 5%, 10%, and 15% Gaussian noise to test the model's robustness to real-world measurement errors [37].
- **Kolmogorov–Smirnov (K-S) Test:** Check the similarity between training and testing data distributions and assess the model's ability to confidently distinguish between classes [38].
- **Model Calibration (Brier Score and Calibration Curves [39], [40]):** Evaluate how well predicted probabilities align with actual outcomes, ensuring reliable confidence estimates.

Together, these tests provide a comprehensive understanding of the model's performance, stability, and real-world readiness.

5.4.1 Model Accuracy

The model was tested over 100 iterations to evaluate mean accuracy, standard deviation, and runtime.

The optimised model achieved a mean accuracy of 90.06%, a 14.33% improvement over the unoptimised baseline. The standard deviation in accuracy reduced to 0.552%, indicating significantly improved model stability and reduced overfitting. Runtime was 115.08 seconds for a full training and evaluation cycle, the substantial accuracy gain justifies the additional computational cost from use of the optimisation algorithms.

	Unoptimised	Final Model
Mean Accuracy (%)	75.73%	90.06%
Change in mean Accuracy (%)	N/A	+14.33%
Standard Deviation in Accuracy (%)	4.857%	0.552%
Change in Standard Deviation of Accuracy (%)	N/A	-4.305%
Model Runtime (s)	0.11	115.08

Table 16 - Classification performance of the final model compared to an unoptimised XGBoost model

At this level of accuracy, the model correctly predicts the HEA phase in approximately 9 out of 10 samples, making it a viable option for practical use. A plot of the accuracy distribution compared to the unoptimised model is provided below in Figure 22.

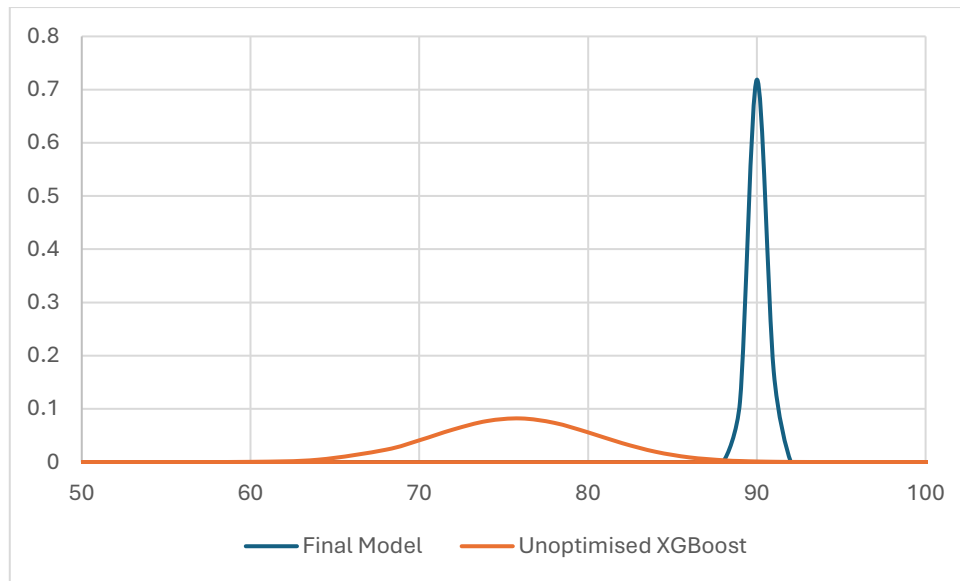


Figure 22 – distribution of accuracy of an unoptimised XGBoost model compared to the final model

5.4.2 Robustness testing with added noise

To simulate real-world measurement noise, Gaussian noise was added at levels of 2%, 5%, 10%, and 15% to the input features. The model was then tested under these conditions to assess its robustness. A resilient model should maintain high accuracy with minimal performance degradation under noisy conditions.

Noise	0%	2%	5%	10%	15%
Mean Accuracy (%)	90.06%	83.16%	80.66%	79.52%	77.62%
Standard deviation of accuracy (%)	0.552%	1.063%	1.177%	1.487%	0.559%

Table 17 – Mean Accuracy and standard deviation with different quantities of gaussian noise added

The model maintained strong performance with added noise, achieving 77.62% mean accuracy under 15% noise. Standard deviation increased slightly at moderate noise levels, reflecting expected minor instability, but remained low overall, confirming the model's robustness and suitability for real-world deployment.

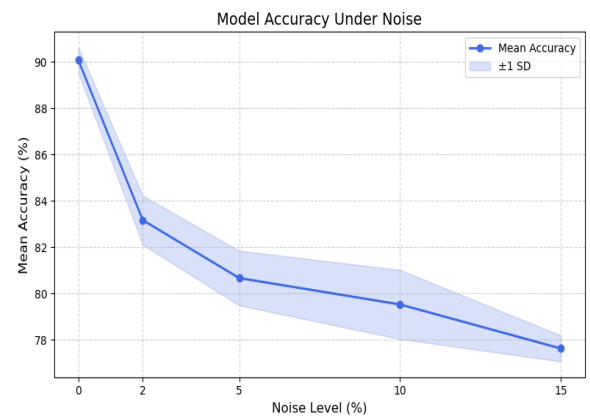


Figure 23 - Mean accuracy and stability of model with different quantities of gaussian noise added

5.4.3 Distribution Alignment Using Kolmogorov–Smirnov Test

The results of the Kolmogorov–Smirnov (K-S) test strongly support the model’s discriminative power and generalisation capability. For predicted probabilities across the three output classes, the K-S statistic was highest for BCC, and lowest for Other. The p-value was less than 0.0001 for all classes, clearly indicating that the model draws a statistically significant distinction between the target and non-target class probabilities. This high separation is visible in Figure 24, the sharp divergence between the two classes reflects the model’s confidence and effectiveness in classifying HEA phases.

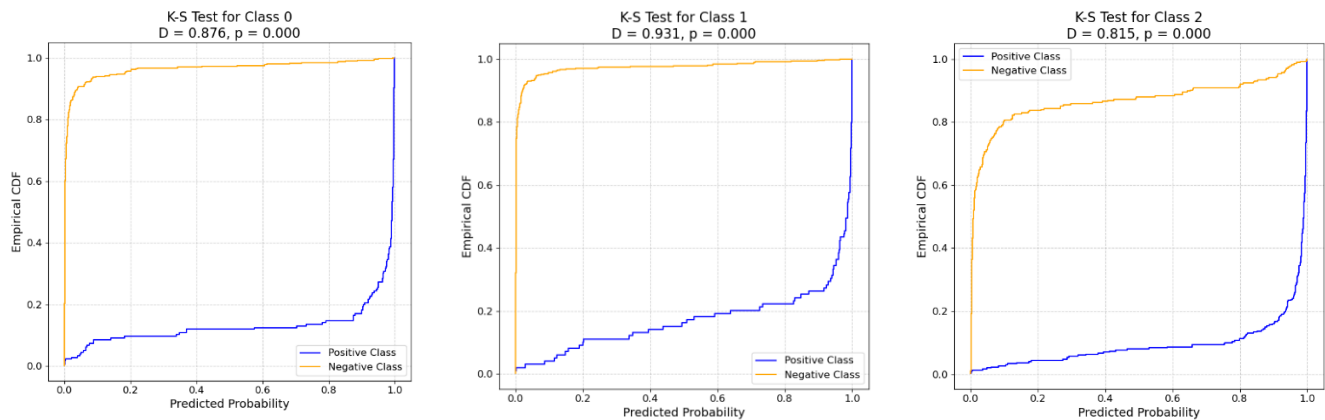


Figure 24 - K-S plots show separation between Positive Class (0:FCC, 1:BCC, 2:Other) and Negative classes

The weaker K-S value for the Other class was likely due to the large variety in amorphous phases, making class distinction more difficult for this group.

The K-S test was also applied to assess the stability of input feature distributions between the training and test datasets (Table 18), with $p > 0.05$ as a threshold. The results showed consistently low K-S statistics and non-significant p-values across all features, with the highest K-S statistic being 0.0607 for the VEC feature. All features exhibited p-values above the 0.05 threshold, with several features reaching a p-value of 1.0, indicating no significant differences in distribution.

Feature	K-S Statistic	P-Value	Distribution Stable
VEC	0.0607	0.0592	True
Nb	0.0439	0.3149	True
density	0.0429	0.3430	True
young_modulus	0.0429	0.3430	True
Ti	0.0418	0.3728	True
Ni	0.0397	0.4371	True
hv	0.0387	0.4714	True
Cr	0.0377	0.5070	True
Ta	0.0324	0.6966	True
Si	0.0063	1.0000	True
Sn	0.0042	1.0000	True

Table 18 - K-S test for features in final model accepted for $p < 0.05$

These findings confirm that there were no major distribution shifts between the train and test sets. Low KS values for feature distributions show consistency between the training and test sets, suggesting the model will perform well on unseen data.

The K-S test results demonstrate both robust class separation and feature distribution consistency, validating the model’s performance and its reliability when applied to unseen data.

5.4.4 Model Calibration with Brier Score and Calibration Curve

Model calibration evaluates how well predicted probabilities match actual outcomes, which is critical for decision-making tasks that depend on confidence estimates. In this analysis, calibration was assessed using Brier Scores and calibration curves.

The Brier Score measures the mean squared difference between predicted probabilities and actual outcomes.

$$BS_c = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Equation 13 – Brier score formula [39]

The Brier Scores for each class were:

- Class 0 (FCC): 0.0438
- Class 1 (BCC): 0.0314
- Class 2 (Other): 0.0748

These low values demonstrate the model is calibrated well. The model is best calibrated for FCC and BCC, with the lowest Brier Scores, while the “Other” class shows notably poorer calibration. This gap may stem from class imbalance due to the relatively large quantity of amorphous phases [39].

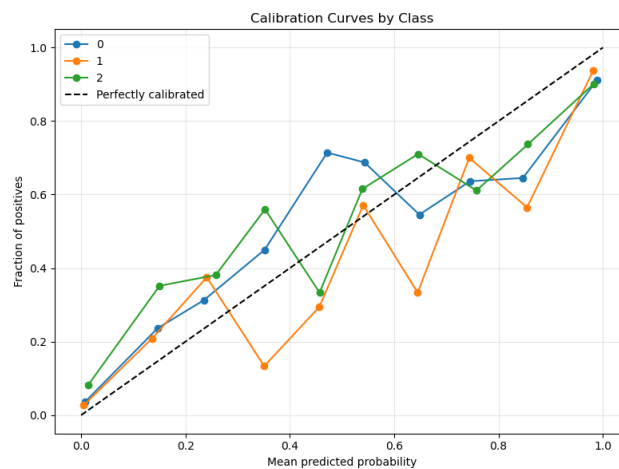


Figure 25 - Calibration Curve of each class

The calibration curves (Figure 25) support these findings [40]. Class 0 (FCC) closely follows the ideal diagonal, with only slight deviation at moderate probabilities. Class 1 (BCC) deviates more noticeably, showing fluctuations around the ideal line. Class 2 (Other) also exhibits moderate instability, with larger deviations that indicate weaker calibration—reflected in its higher Brier Score.

Despite some deviations, all classes exhibit a positive trend, suggesting that the model has successfully learned meaningful patterns from the data. This is supported by both the calibration plots and Brier Scores, which indicate that the model is reasonably well-calibrated and lends some credibility to its predictions. However, observed fluctuations imply a degree of uncertainty—potentially stemming from class overlap. As a best practice, it would be

advisable to apply additional scrutiny to low confidence predictions, potentially through DFT evaluations.

5.5 Discussion on project success and revisiting aims

The project successfully met its objectives by developing a modular machine learning framework that enables fast and dynamic model development, integrating data encoding, additional thermodynamic-fields, feature selection, hyperparameter optimisation, and detailed validation methods. The model supports any form of continuous data, opening up additional numerical fields for use within the model in future research.

Various optimisation methods were integrated and compared, leading to an engineered phase prediction model that achieved a mean accuracy of 90.06% over 100 independent validation cases, with a standard deviation of 0.552. A full training cycle completing in 115 seconds on standard laptop hardware, demonstrating the model's suitability for practical engineering applications.

The model demonstrated high robustness to added noise, use of a K-S test demonstrated strong class separation and generalisability of the model, and calibration tests showed the model has found meaningful patterns in the data.

5.6 Model Limitations

While the machine learning framework achieved high classification accuracy, several limitations affect its broader applicability.

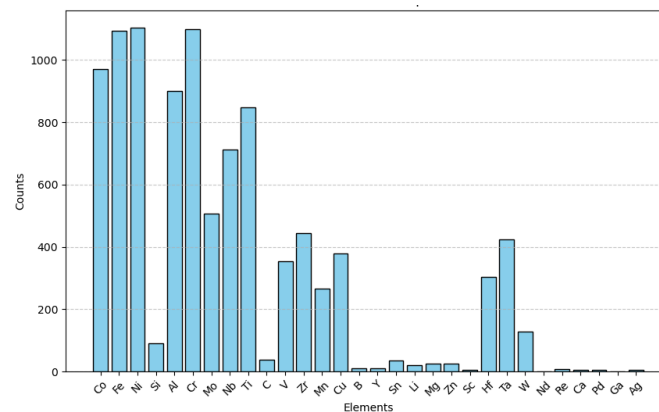


Figure 26 – Number of appearances per element in datasets 1 and 2 (visible in Table 2)

The dataset is biased toward well-studied HEA systems, with an imbalance in distribution of alloying elements, visible in Figure 26. This limits the model's ability to generalise underrepresented elements. More training fields in these elemental regions would be required to improve model generalisability.

Furthermore, the features used are derived purely from compositional data mechanical performance and basic thermodynamic estimates, omitting effects from processing conditions (e.g., annealing, quenching) and structural factors (e.g., grain size, lattice distortion). These simplifications reduce the model's ability to capture real-world phase behaviour in complex environments.

Calibration tests revealed some fluctuations in model confidence. In a research context, performing additional DFT analysis on predictions with confidence scores below 0.7 would be good practice. However, in a development setting, this approach remains more efficient than traditional methods. It reduces the need for costly testing by allowing the model to provide an initial evaluation and generate a prioritised set of candidate predictions.

Finally, the use of ensemble methods like XGBoost provides high performance but limited interpretability. While feature importance rankings offer some insight, the model remains largely a black box, which may hinder its adoption in settings that require explainable decision.

6. Conclusions

The work presented here introduces a high-performance, modular machine-learning framework for predicting phase structures in HEAs, offering a scalable, data-driven alternative to costly trial-and-error experiments and simulations. By integrating thermodynamic feature engineering, advanced optimisation algorithms, and data-imputation through the CEP algorithm—the model reliably surpasses 90 % classification accuracy on an unseen test set of 198 entries. A full training cycle completes in 115 seconds on standard laptop hardware, enabling rapid iteration and deployment in engineering workflows.

Despite these strengths, several limitations persist. The training dataset is biased toward transition-metal systems, which may hinder predictive accuracy for novel or underrepresented elemental combinations. Predictions are based solely on compositional and derived thermodynamic features, omitting contextual factors such as processing history and microstructural variables. While CEP imputation broadens coverage, the resulting values introduce uncertainty. Additionally, ensemble methods like XGBoost deliver high performance at the expense of interpretability, potentially limiting adoption in settings that demand transparent decision support.

7. Future Work

Looking forward, this framework lays a solid foundation for accelerated HEA discovery, especially in high-stakes domains such as aerospace materials design. Future work involves expanding the dataset with underrepresented elements, incorporating processing and structure-related descriptors, and exploring more advanced modelling techniques to enhance interpretability without sacrificing accuracy. These efforts will drive smarter, faster alloy development and underscore the transformative potential of machine learning in materials

References

- [1] E. P. George, D. Raabe and R. O. Ritchie, “High-entropy alloys,” *Nature Reviews | Materials*, vol. 4, pp. 515-534, 2019.
- [2] J.-W. Yeh, S.-K. Chen, S.-J. Lin, J.-Y. Gan, T.-S. Chin, T.-T. Shun, C.-H. Tsau and S.-Y. Chang, “Nanostructured High-Entropy Alloys with Multiple Principal Elements: Novel Alloy Design Concepts and Outcomes,” *Advanced Engineering Materials*, vol. 6, no. 5, pp. 299-303, 2004.
- [3] Q. Guo, X. Xu, X. Pei, Z. Duan, P. K. Liaw, H. Hou and Y. Zhao, “Predict the phase formation of high-entropy alloys,” *Journal of Materials Research and Technology*, vol. 22, no. 2238-7854, pp. 3331-3339, 2023.
- [4] K. LI and H. S. Jason, “DFT dataset for high entropy materials [Data set],” *Journal of Materials Chemistry*, vol. 12, no. 21, p. 12412–12422, 2024.
- [5] S. B. Kotsiantis, “Feature selection for machine learning classification problems: a recent overview,” *Artificial Intelligence Review*, vol. 42, pp. 1-20, 2011.
- [6] Y. Zhang and C. Ling, “A strategy to apply machine learning to small datasets in materials science,” *npj Computational Materials*, vol. 25, pp. 1-9, 2018.
- [7] M. Dada, P. Popoola, S. Adeosun and N. Mathe, “High Entropy Alloys for Aerospace Applications,” in *Aerodynamics*, Intech Open, 2021.
- [8] X. Gao, R. Chen, T. Liu, H. Fang, G. Qin, Y. Su and J. Guo, “High-entropy alloys: a review of mechanical properties and deformation mechanisms at cryogenic temperatures,” *Journal of Materials Science*, vol. 57, p. 6573–6606, 2022.
- [9] F. Otto, Y. Yang, H. Bei and E. P. George, “Relative effects of enthalpy and entropy on the phase stability,” *Acta Materialia*, vol. 61, no. 7, pp. 2628-2638, 2013.
- [10] S. Jain, R. Jain, K. R. Rao and A. Bhowmik, “Leveraging machine learning to minimize experimental trials and predict hot deformation behaviour in dual phase high entropy alloys,” *Materials Today Communications*, vol. 41, pp. 1-9, 2024.
- [11] S. Singh, N. K. Katiyar, S. Goel and S. N. Joshi, “Phase prediction and experimental realisation of a new high entropy alloy using machine learning,” *Scientific Reports*, vol. 13, no. 4811, pp. 1-17, 2023.
- [12] Y. Yan, K. Hu, Y. Liu, Y. Zhu, W. Li and T. Zhu, “Recent machine learning-driven investigations into high entropy alloys: A comprehensive review.,” *Journal of Alloys and Compounds*, vol. 1010, no. 177823, pp. 1-20, 2025.

- [13] I. Peivaste, E. Jossou and A. A. Tiamiyu, "Data-driven analysis and prediction of stable phases for high-entropy alloy design.," *Scientific Reports*, vol. 13, no. 22556, pp. 1-21, 2023.
- [14] X. Liu, J. Zhang and Z. & Pei, "Machine learning for high-entropy alloys: Progress, challenges and opportunities.," *Progress in Materials Science* , vol. 131, no. 101018, pp. 1-33, 2023.
- [15] D. Kochkova, J. A. Smith, A. Alievaa, Q. Wanga, M. P. Brenner and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *PNAS*, vol. 118, no. 21, pp. 1-8, 2021.
- [16] M. Ryo and M. C. Rillig, "Statistically reinforced machine learning for nonlinear patterns and variable interactions," *Emerging Technologies*, vol. 8, no. 11, pp. 1-16, 2017.
- [17] W. Huang, P. Martin and H. L. Zhuang, "Machine-learning phase prediction of high-entropy alloys," *Acta Materialia*, vol. 169, pp. 225-236, 2019.
- [18] A. Ul Hassan, M. S. Khan and M. Shah, "Comparison of Machine Learning Algorithms in Data classification," in *24th International Conference on Automation & Computing*, Newcastle Upon Tyne, 2018.
- [19] S. Tahosin, A. Sheakh, T. Islam and R. J. Lima, "Optimizing brain tumor classification through feature selection and hyperparameter tuning in machine learning models," *Informatics in Medicine Unlocked*, vol. 43, no. 101414, pp. 1-9, 2023.
- [20] Amazon, Amazon Machine Learning Developer Guide, Amazon, 2025.
- [21] K. Weiss, T. M. Khoshgoftaar and D. Wang, "A survey of transfer learning," *Journal of Big data*, vol. 3, no. 9, pp. 1-40, 2016.
- [22] C. Borg, C. Frey, J. Moh, T. Pollock, S. Gorsse, D. Miracle, O. Senkov, B. Meredig and J. Saal, "Expanded dataset of mechanical properties and observed phases of multi-principal element alloys [dataset]," *Scientific Data*, vol. 7, 2020.
- [23] X. Chang, M. Zeng, K. Liu and L. Fu, "Phase Engineering of High Entropy Alloys," *Advanced Materials*, vol. 32, no. 1907226, pp. 1-22, 2020.
- [24] W. Siedlecki and J. Sklansky, "A note on genetic algorithms for large-scale feature selection," *Pattern Recognition Letters*, vol. 10, no. 5, pp. 335-347, 1989.
- [25] G. Luo, "A review of automatic selection methods for machine learning algorithms and hyper-parameter values," *Network Modeling Analysis in Health Informatics and Bioinformatics*, vol. 5, no. 18, pp. 1-16, 2016.

- [26] C. Erden, “Genetic algorithm-based hyperparameter optimization of deep learning models for PM2.5 time-series prediction,” *International Journal of Environmental Science and Technology*, vol. 20, pp. 2959-2982, 2023.
- [27] Mathworks, “Tune Experiment Hyperparameters by Using Bayesian Optimization,” Mathworks, [Online]. Available: <https://uk.mathworks.com/help/deeplearning/ug/tune-experiment-hyperparameters-using-bayesian-optimization.html?requestedDomain=>. [Accessed 2025 04 30].
- [28] S. Gorsse, M. Nguyen, O. Senkov and D. Miracle, “Database on the mechanical properties of high entropy alloys and complex concentrated alloys [dataset],” *Data in Brief*, vol. 21, no. 10, 2018.
- [29] Spotfire, “What is a random forest?,” Spotfire, [Online]. Available: <https://www.spotfire.com/glossary/what-is-a-random-forest>. [Accessed 08 April 2025].
- [30] IBM, “What is random forest?,” IBM, [Online]. Available: <https://www.ibm.com/think/topics/random-forest#:~:text=Random%20forest%20is%20a%20commonly,both%20classification%20and%20regression%20problems..> [Accessed 29 April 2025].
- [31] F. Omarzai, “XGBoost classification in depth,” Medium, 21 July 2024. [Online]. Available: <https://medium.com/@fraidoonomarzai99/xgboost-classification-in-depth-979f11ef4bf9>. [Accessed 8 April 2025].
- [32] S. Jaiswal, “Multilayer Perceptrons in Machine Learning: A Comprehensive Guide,” Datacamp, 05 April 2025. [Online]. Available: <https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning>. [Accessed 08 April 2025].
- [33] GoodmanSciences, “Periodic Table of Elements.csv,” 6 June 2017. [Online]. Available: <https://gist.github.com/GoodmanSciences/c2dd862cd38f21b0ad36b8f96b4bfl ee>. [Accessed 29 April 2025].
- [34] Massachusetts Institute of Technology, “Big O Notation,” [Online]. Available: https://web.mit.edu/16.070/www/lecture/big_o.pdf. [Accessed 2025 05 01].
- [35] IBM, “What is LASSO Regression,” 18 January 2024. [Online]. Available: <https://www.ibm.com/think/topics/lasso-regression#:~:text=Lasso%20regression%20is%20a%20regularization,regularization%20for%20linear%20regression%20models..> [Accessed 30 April 2025].

- [36] M. Ogunsanya, J. Isichei and S. Desai, “Grid search hyperparameter tuning in additive manufacturing processes,” *Manufacturing Letters*, vol. 35, pp. 1031-1042, 2024.
- [37] C. Long, W. Zhang, L. Ruihui, H. Wang and Z. Dong, “PC2 -PU: Patch Correlation and Point Correlation for Effective Point Cloud Upsampling; Yang, Bisheng,” Lisbon, 2022.
- [38] I. M. C. R. G. Laha and a. J. Roy, Handbook of methods of applied statistics, New York: John Wiley and sons, 1967.
- [39] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *U. S. Weather Bureau*, vol. 78, no. 1, pp. 1-3, 1950.
- [40] L. Prichard and V. J. Barwick, “Preparation of Calibration Curves A Guide to Best Practice,” LGC Group, 2003.

Appendix – Algorithm Pseudocode

Model Class

```
1. CLASS Model:
2.   METHOD __init__(x_columns=None, thermo_fields=False, feature_selection=None, y_column='phase', file_loc='...',
model_type='xgb', fill=False, params=None, hp_optimisation=None)
3.   SET parameters as attributes
4.   IF x_columns is None:
5.     LOAD data file
6.     EXTRACT column names as x_columns
7.   REMOVE target column from x_columns
8.   IF thermo_fields is True:
9.     APPEND thermodynamic fields to x_columns
10.  STORE unfiltered and filtered x_columns
11.  SET y_column, model_type, file_df
12.  CALL create_ml_df()
13.  EXTRACT X and y from ml_df
14.  IF feature_selection is 'elliott':
15.    CALL run_elliott_fs()
16.  IF feature_selection is 'genetic':
17.    CALL run_genetic_fs()
18.  CALL create_model() and assign to self.model
19.  IF hp_optimisation is 'bayesian':
20.    RUN bayesian_optimization() → update params → recreate model
21.  ELIF hp_optimisation is 'genetic':
22.    RUN genetic_hyperparameter_optimization() → update params → recreate model
23.  ELIF hp_optimisation is 'grid':
24.    RUN grid_search_hyperparameter_optimization() → update params → recreate model
25.  ELIF hp_optimisation is 'LASSO':
26.    parameters['regularisation'] = 'L1'
25.  METHOD run_elliott_fs()
26.    PRINT "Running elliott fs"
27.    FILTER columns (remove target phase labels)
28.    CREATE ElliotFeatureSelection object
29.    FIT to data
30.    UPDATE selected x_columns

31.  METHOD run_genetic_fs()
32.    PRINT "Running genetic fs"
33.    PREPARE column headers
34.    CREATE GeneticFeatureSelector
35.    RUN selection
36.    APPEND elemental features
37.    UPDATE x_columns

38.  METHOD create_ml_df()
39.    COMBINE x_columns and y_column
40.    REMOVE thermo features temporarily
41.    CREATE training dataframe
42.    IF thermo features were removed:
43.      ADD them back in
44.    RETURN processed dataframe

45.  METHOD create_model()
46.    TRY:
47.      PREPARE train_df with x_columns + y_column
48.    EXCEPT:
49.      INCLUDE phase one-hot if needed
50.    MATCH model_type:
51.      CASE 'xgb':
52.        CALL create_xgb_model() with or without params
53.      CASE 'mlp':
54.        CALL create_mlp_model() with or without params
55.      CASE 'rf':
56.        CALL create_random_forest_model() with or without params
57.    STORE train_df and score
58.    RETURN model
```

Genetic Feature Selection

```
01. INPUT: X, y, y_column, model_type, n_generations, population_size, mutation_rate, crossover_rate

02. SET task_type ← 'classification' IF y_column == 'phase' ELSE 'regression'

03. INITIALIZE DEAP base classes:
04.   CREATE FitnessMax (maximize objective)
05.   CREATE Individual (a list with FitnessMax)

06. FUNCTION generate_individual(model_type):
07.   IF model_type == 'rf':
08.     RETURN [rand(10,200), rand(1,20), rand(2,20)]
09.   ELSE IF model_type == 'xgb':
10.     RETURN [rand(10,200), rand(1,20), rand_float(0.01,0.3)]
11.   ELSE IF model_type == 'mlp':
12.     RETURN [rand(32,128), rand(16,64), rand(8,32), rand_float(1e-5,1e-1), rand_float(0.001,0.1)]

13. FUNCTION evaluate(individual, X, y, model_type, task_type):
14.   SET params ← decode individual into model hyperparameters
15.   IF model_type == 'rf' → RandomForest; IF 'xgb' → XGBoost; IF 'mlp' → MLP
16.   PERFORM 5-fold cross-validation with model on (X, y)
17.   RETURN mean accuracy (or neg-RMSE for regression)

18. FUNCTION custom_mutate(individual, model_type, indpb):
19.   FOR gene in individual:
20.     IF rand() < indpb:
21.       Mutate gene based on model_type
22.   RETURN mutated individual

23. FUNCTION setup_toolbox():
24.   REGISTER: gene generators, individual creation, population, evaluate, crossover, mutate, selection
25.   RETURN toolbox

26. FUNCTION run_evolution(toolbox, population, n_generations, cxpb, mutpb):
27.   FOR generation in 1 to n_generations:
28.     SELECT parents via tournament
29.     APPLY crossover and mutation to generate offspring
30.     EVALUATE all offspring
31.     REPLACE population with next generation
32.   RETURN best individual in final generation

33. FUNCTION extract_best_params(best_individual, model_type):
34.   MAP gene list to hyperparameter dictionary
35.   RETURN best_params

36. MAIN PROCEDURE:
37.   toolbox ← setup_toolbox()
38.   population ← generate initial population
39.   best_individual ← run_evolution(toolbox, population, n_generations, crossover_rate, mutation_rate)
40.   best_params ← extract_best_params(best_individual, model_type)
41.   OUTPUT best_params
```

Bayesian Search

```
01. FUNCTION MLPWrapper:
02.   INIT with layer sizes, alpha, learning_rate_init, and classification flag
03.   FUNCTION fit(X, y): trains MLPClassifier or MLPRegressor based on flag
04.   FUNCTION predict(X): return model prediction
05.   FUNCTION score(X, y): return model score

06. FUNCTION get_param_space(model_type):
07.   RETURN search space dictionary for RF, MLP, or XGB

08. FUNCTION get_model(model_type, is_classification):
09.   RETURN corresponding estimator based on type and task

10. FUNCTION is_classification_task(y):
11.   IF target type is binary or multiclass → RETURN True
12.   IF y is float OR has >20 unique values → RETURN False
13.   ELSE RETURN True

14. FUNCTION bayesian_optimization(model_type, X, y, cv, n_iter, random_state):
15.   SET is_classification ← is_classification_task(y)
16.   SET scoring ← 'accuracy' if classification else 'r2'
17.   model ← get_model(model_type, is_classification)
18.   param_space ← get_param_space(model_type)

19.   IF model or param_space is None → RAISE ValueError

20.   Flatten y (ravel)
21.   IF classification:
22.     ENCODE y using LabelEncoder

23.   TRY:
24.     RUN BayesSearchCV(model, param_space, scoring, cv)
25.   EXCEPT:
26.     INVERT classification flag and re-run with new model and scoring

27.   EXTRACT best_params from bayes_search
28.   IF model_type == "mlp":
29.     FORMAT params as hidden_layer_sizes, alpha, learning_rate_init
30.   ELSE:
31.     RETURN best_params
```

Grid Search

```
01. FUNCTION evaluate(individual, X, y, model_type, task_type, cv=5):
02.   Flatten y to 1D
03.   IF model_type == 'rf':
04.     CREATE RandomForestClassifier/Regressor with parameters from individual
05.   ELIF model_type == 'xgb':
06.     CREATE XGBClassifier/Regressor with parameters from individual
07.   ELIF model_type == 'mlp':
08.     CREATE MLPClassifier/Regressor with layer sizes, alpha, learning_rate
09.   PERFORM cross_val_score with chosen model
10.   RETURN mean score from cross-validation

11. FUNCTION grid_search_hyperparameter_optimization(X, y, y_column, model_type='rf', cv=2):
12.   IF y_column is 'phase':
13.     SET task_type to 'classification'
14.   ELSE:
15.     SET task_type to 'regression'
16.   Flatten y to 1D

17.   IF model_type == 'rf':
18.     DEFINE param_grid for RandomForest (n_estimators, max_depth, min_samples_split)
19.     model ← RandomForestClassifier or Regressor
20.   ELIF model_type == 'xgb':
21.     DEFINE param_grid for XGBoost (n_estimators, max_depth, learning_rate)
22.     model ← XGBClassifier or Regressor
23.   ELIF model_type == 'mlp':
24.     DEFINE param_grid for MLP (hidden_layer_sizes, alpha, learning_rate_init)
25.     model ← MLPClassifier or Regressor (max_iter=500)

26.   CREATE GridSearchCV with model and param_grid
27.   Fit GridSearchCV on data (X, y)
28.   PRINT best parameters
29.   RETURN best parameters
```

Genetic Hyperparameter Optimisation

```
01. FUNCTION evaluate(individual, X, y, model_type, task_type, cv=5):
02.   Flatten y into 1D array
03.   IF model_type is 'rf':
04.     SET model as RandomForestClassifier/Regressor with params from individual
05.   ELIF model_type is 'xgb':
06.     SET model as XGBClassifier/Regressor with params from individual
07.   ELIF model_type is 'mlp':
08.     SET model as MLPClassifier/Regressor with hidden layers and learning params
09.   RETURN mean cross-validation score using model

10. FUNCTION custom_mutate(individual, model_type, indpb):
11.   FOR each hyperparameter in individual:
12.     With probability indpb, randomly mutate its value within bounds
13.   RETURN mutated individual

14. FUNCTION genetic_hyperparameter_optimization(X, y, y_column, model_type, n_generations, population_size, mutation_rate,
crossover_rate):
15.   DETERMINE task_type as 'classification' or 'regression' based on y_column
16.   INITIALIZE DEAP evolutionary setup:
17.     CREATE fitness function (maximize)
18.     CREATE individual class as list of hyperparameters
19.     DEFINE parameter ranges for selected model_type
20.     REGISTER functions for population, individual, evaluation, mutation, selection
21.   INITIALIZE population with population_size
22.   RUN evolutionary algorithm for n_generations:
23.     APPLY selection, crossover, mutation, evaluation
24.   SELECT best_individual from population
25.   EXTRACT and FORMAT best_individual's parameters as dictionary
26.   PRINT best parameters
27.   RETURN best_params
```